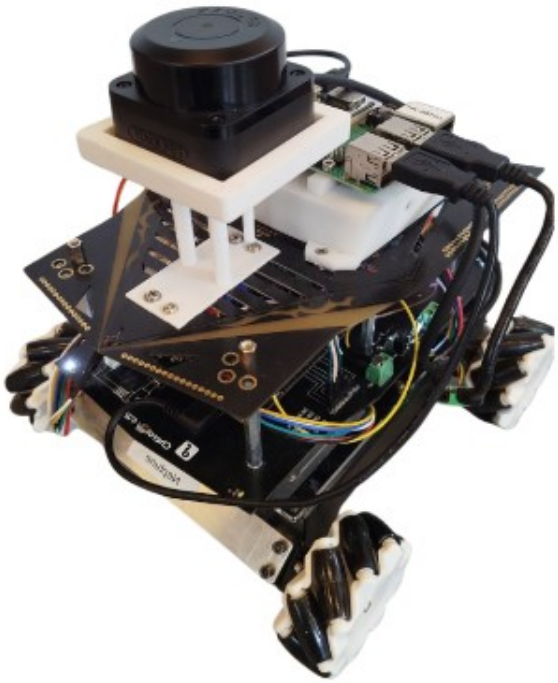
CentraleSupélec

Université Paris-Saclay

CentraleSupélec
Pôle Projet Robotique

Plateforme Mobile – Rapport Intermédiaire



Membres du groupe :
- Louis HEBRANT
- Siméon KOCH
- Quentin MININGER
- Jean-Baptiste SENET KRAKOWIAK
- Alois STOLTZ

Date : 04 juin 2025

Encadrant : Morgan Roger

Année universitaire 2024–2025

Table des matières

1 Introduction **1**

2 Project Overview **2**
2.1 Hardware and System Design . . . 2
2.1.1 System Architecture . . . 2
2.1.2 Raspberry Pi . . . 3
2.1.3 Cherokey PCB . . . 3
2.1.4 RoMeo BLE Board . . . 4
2.1.5 Sensors and Features . . . 4
2.1.6 Motors and Feedback Systems . . . 5
2.2 Initial setup and legacy code review . . . 5
2.2.1 Initial state of the project . . . 5
2.2.2 Project's objectives and Minimal Viable Product . . . 6

3 Speed Control of Wheel Rotation **7**

4 Current Probe and Electrical Analysis of the Motor **9**
4.1 Motivation . . . 9
4.2 Measurement of Angular Acceleration ($\frac{d\omega}{dt}$) . . . 9
4.2.1 Characteristic time of Wheel Speed Dynamics . . . 10
4.2.2 Characteristic Time of Sensor Feedback . . . 10
4.2.3 Reducing the Characteristic Time of Sensor Feedback . . . 10
4.3 Motor Analysis . . . 11
4.4 Signal Characteristics . . . 12
4.5 Current Probe Implementation . . . 13
4.6 Power Supply Issues . . . 13
4.7 Signal Conditioning Prior to the Instrumentation Amplifier . . . 14
4.8 Remaining Tasks . . . 16

5 Obstacle Detection **18**
5.1 Initial Sensor Layout . . . 18
5.2 LiDAR Decision . . . 18
5.3 Physical and Electronic Integration . . . 19
5.4 Data Acquisition (Software integration) . . . 20
5.5 Landmark Detection using RANSAC . . . 23
5.6 Landmark detection using convex hull generation . . . 24
5.7 Controlled Movement : Translation and Rotation . . . 26
5.7.1 Forward + distance . . . 26
5.7.1.1 Distance measurement . . . 27
5.7.2 Turn + angle . . . 28
5.7.2.1 Measurement of the angle of rotation (First method) . . . 29
5.7.2.2 Measurement of the angle of rotation (Second method) . . . 29
5.7.3 Translation to the right or left by a given distance . . . 31
5.8 Obstacle avoidance with lidar and the first steps of an Autonomous Navigation . . . 32
5.9 Conclusion of this part : Towards a Robust LiDAR-Based SLAM for Autonomous Navigation . . . 33

6 Conclusion **34**
6.1 Conclusion of the report . . . 34
6.2 Remaining challenges . . . 34

Chapitre 1

Introduction

Autonomous navigation is a key component in contemporary robotics, allowing machines to move and adapt to their surroundings without human input. This functionality depends on the seamless integration of sensors, control algorithms, and hardware elements, enabling precise and efficient movement. In this project, we build upon the work of a previous team with the goal of further enhancing the performance of a four-wheeled robotic platform. The robot features a Raspberry Pi, programmed in Python, which handles high-level processing and control ; a Romeo BLE board, programmed via Arduino, which oversees motor control and sensor interactions ; and a Cherokey PCB, responsible for power distribution and motor coordination.

The prior team concentrated on improving the speed control system to achieve smoother and more consistent motion. By implementing a PID controller, they addressed some inefficiencies stemming from the absence of time-step consideration, which led to more reliable and linear movement. They also began developing a wheel position control system to enhance the robot's turning capabilities. Despite these advancements, some issues persisted. The motors exhibited non-linear behavior, and consistency between the four wheels remained imperfect. Additionally, despite preliminary works on the LiDAR, full integration of the sensor was not completed. Building on previous work, our objective is to overcome these limitations and boost the robot's navigation performance.

Our efforts focus on refining the closed-loop control system to ensure stable and precise movement, especially during turning. By improving the time-step calculation method using incremental encoders, we aim to increase wheel accuracy at lower speeds, ultimately allowing the robot to execute more controlled and accurate turns.

Since the communication with the LiDAR has been established by the previous group, our group seeks to connect the sensor with the robot. This upgrade will possibly allow for more accurate and detailed environmental mapping, significantly improving the robot's ability to detect and respond to obstacles in real time. The enhanced detection capabilities will enable the robot to adapt its trajectory dynamically and maneuver effectively in complex environments.

This report presents the methods and strategies employed to reach these objectives, along with the technical progress achieved over the course of the project.

Chapitre 2

Project Overview

2.1 Hardware and System Design

Given that the description and summary of the robot's hardware and platform were already clearly and effectively presented by the other group, we chose to reuse parts of the beginning of the 2024 report [1] with only minor changes, mainly in the Sensors and Features and Motors and Feedback Systems sub-sections, for the beginning of this section.

2.1.1 System Architecture

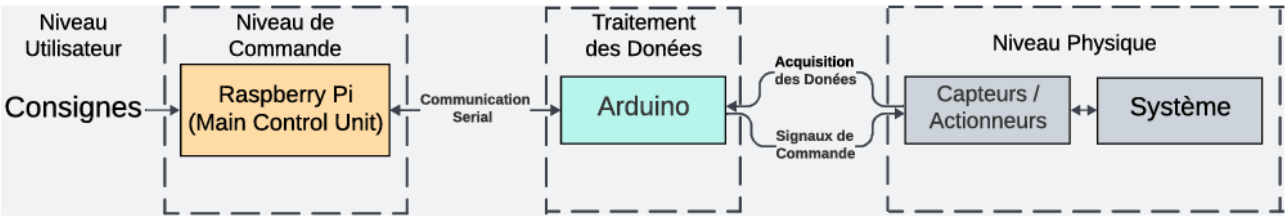


FIGURE 2.1 – System Architecture

The motor system of the robot, part of the "Système" block mentioned above, is a critical component responsible for enabling its movement and overall mobility. This system is composed of four omnidirectional wheels, each driven by indivual motors controlled by a combination of the Arduino Uno and the Cherokey PCB. The omnidirectional wheels should allow for flexible motion, including straight movement, rotation, and diagonal paths, making the robot capable of navigating complex environments with precision.

The Arduino Uno acts as the intermediary control board, receiving high-level commands from the Raspberry Pi via serial communication and translating them into motor control signals. These signals are then transmitted to the Cherokey PCB, which distributes power to the motors and ensures their operation is synchronized.

Encoders connected to the wheels provide feedback on the motor's performance, such as speed and position. This feedback is crucial for implementing closed-loop control, which adjusts motor operations in real-time to ensure precise movements and trajectory execution. In this section, we will detail each component of the motor system, explaining its role and contribution to the overall functionality of the robot's movement and navigation capabilities.

2.1.2 Raspberry Pi

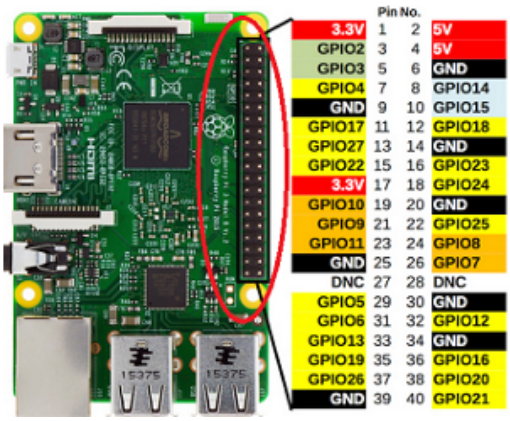


FIGURE 2.2 – Raspberry Pi Pins

The Raspberry Pi 3 Model B+ is a powerful single-board computer that serves as the central processing unit for the robot. With its 1.4GHz 64-bit quad-core processor, wireless LAN, Bluetooth, and Power-over-Ethernet support, it provides the computational power and connectivity to control the robot effectively. Its compact design and extensive functionality make it an ideal choice for robotics projects.

In this project, the Raspberry Pi acts as the director of the robot, responsible for performing complex calculations and communicating commands to other components, such as the Arduino ROMEO board, via serial communication. It is chosen for its higher processing capabilities compared to the Arduino, enabling it to handle tasks that require significant computational resources, such as running Python programs and managing sensor data.

It features multiple ports and connections, including HDMI for screen connectivity, USB ports for peripherals, a camera port for vision integration, and GPIO pins for interfacing with other electronic components. Additionally, the SD card inserted into the Raspberry Pi serves as its hard drive, containing the operating system and all necessary Python and Arduino libraries. It is crucial to the robot's operation, as removing or replacing the card would result in the loss of essential files.

To interact with the Raspberry Pi, we use VNC (Virtual Network Computing), which allows remote desktop access via Ethernet or Wi-Fi. This setup enables control and programming of the robot without needing a physical monitor or keyboard connected to the Raspberry Pi.

In this project, we will be working with two Raspberry Pi boards to develop and test functionalities related to the LiDAR system. However, under normal operating conditions, only one Raspberry Pi will be attached to the robot to perform its primary functions. This dual setup during development allows us to isolate and optimize the LiDAR-based obstacle detection and navigation capabilities before integrating them fully into the robotic system.

Remote desktop access is enabled via VNC over Ethernet or Wi-Fi. For development, two Raspberry Pi boards were used to test LiDAR functionality. In the final version, only one will remain on the robot.

2.1.3 Cherokey PCB

The Cherokey PCB is a critical component in the robot's architecture, responsible for controlling the four motors that drive the robot's movement. It is equipped with two H-bridges, which allows control over both the voltage applied and the direction of rotation for only the two motors at the front of the robot. The other two H-bridges are located directly on the RoMeo board 2.1.4.

The Cherokey PCB cherokey pcb is powered by a battery and controls four motors through commands from the Romeo Card. It features various pins and connections that provide flexibility for integrating additional components or adapting the system to specific project needs. The disadvantage of this board is that the H-Bridge ports are connected in pairs, meaning the four ports cannot be controlled independently.

The H-bridges enable :

— **Voltage Control (PWM) :** By utilizing Pulse Width Modulation (PWM), the Cherokey PCB regulates the average voltage delivered to the motors. This is achieved by adjusting the ratio of time spent at full power versus no power. For example, if the system operates at 50% power (50% at 0V and 50% at 5V), the average voltage supplied is 2.5V. This allows for smooth control over the motor speed.

— **Direction Control :** By switching voltage polarity, the motors can rotate forward or backward.

2.1.4 RoMeo BLE Board

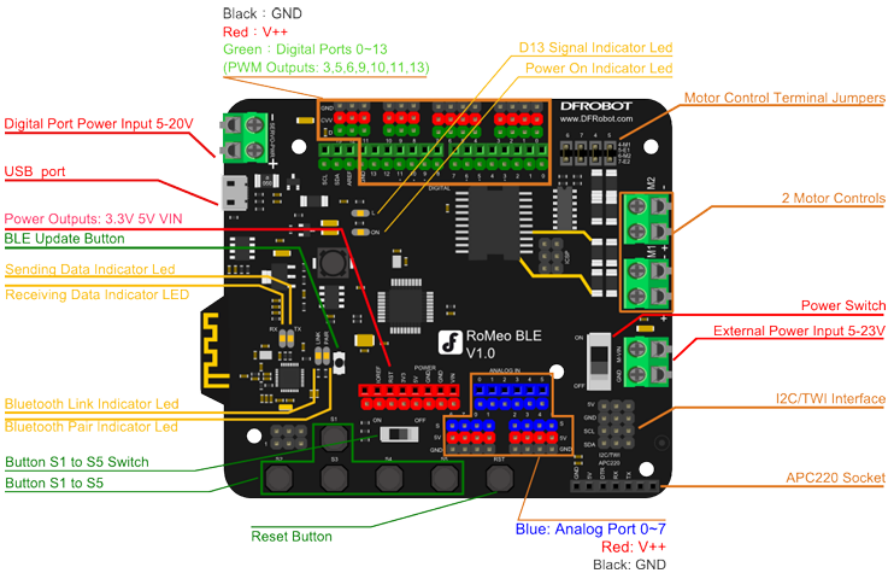


FIGURE 2.3 – RoMeo BLE Board

The RoMeo BLE Board is a microcontroller board that combines the functionality of an Arduino with additional features such as motor drivers, sensor integration, and communication modules. Similar to the Cherokey PCB, the RoMeo BLE features two independent H-Bridge power ports, enabling it to drive two motors directly. In this setup, the RoMeo board controls the robot's four motors and servomotors, while also receiving data from various sensors, including an ultrasonic sensor, a LiDAR. The LiDAR data is particularly crucial for precise distance measurements and mapping. All sensor data is processed by the Raspberry Pi, which then sends motor and servo control commands back to the RoMeo, ensuring smooth coordination.

Complementing the RoMeo board with the Cherokey PCB provides significant advantages. The Cherokey PCB offers additional motor-driving capabilities and can share the same power source with the RoMeo, creating a robust and unified system for powering motors and peripherals. This pairing not only enhances the robot's power management but also allows for more complex configurations, such as controlling additional motors or actuators.

2.1.5 Sensors and Features

The primary sensor for obstacle detection in the robot is the URM37 ultrasonic sensor, mounted on a servomotor for directional scanning. It operates by emitting ultrasonic pulses and measuring the time of flight (ToF) of the reflected sound waves to calculate distances.

The ultrasonic sensor is connected to the Romeo BLE board (Arduino), which processes the measured distances and sends the data to the Raspberry Pi via serial communication. The servo motor enables the sensor to scan objects in multiple directions, increasing its field of detection. It provides linear distance measurements for obstacles directly in its path, and it is primarily used to detect static objects and trigger trajectory adjustments to avoid collisions.

In addition, the robot architecture includes a camera module connected to the Raspberry Pi, which was originally intended for advanced navigation tasks, such as object recognition and mapping using computer vision. However, the camera was disabled by one of the previous group because of a high computational load of image processing.

The RPLIDAR C1 is a LiDAR sensor (Light Detection and Ranging). It detects obstacles and enables mapping. It emits electromagnetic waves and measures the time it takes for the reflected signal to return to the sensor.

One of its advantages is that it scans 360 degrees with a high sampling rate (5,000 measurements per second). It provides accurate distance measurements over a range of over 10 meters, making it useful for SLAM mapping.

The RPLIDAR C1 operates via a serial connection and we used it with the Raspberry Pi to process and we analyze data in real time using Python.

2.1.6 Motors and Feedback Systems

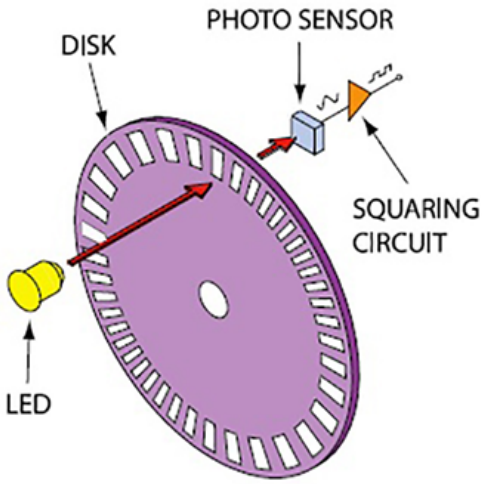


FIGURE 2.4 – Encoder Structure

The motors used in the robot integrate the encoder, gearbox, and motor into a single unit, simplifying assembly and providing compact functionality. These geared motors are equipped with two toothed wheels, allowing precise control of the output shaft through the encoder. The encoders specifically measure the rotation of the output shaft after the gearbox, providing accurate feedback about the motor's speed and position. This information is essential for precise movement control and allows the robot to perform tasks like maintaining a straight trajectory or executing complex maneuvers.

The motors are powered through the H-Bridge power ports provided by the Cherokey PCB and the RoMeo BLE Board. The RoMeo board sends PWM signals to control motor speed and directional commands, ensuring smooth and efficient motor operation for robotic applications.

The encoders in our motors consist of two toothed wheels with a slight offset, which are essential for measuring both the rotation speed and direction. Each encoder has two channels, and by detecting transitions from a low state to a high state on one channel, the system can determine motor movement. The direction of rotation is identified by observing which channel changes state first, while the speed is calculated based on the frequency of these state changes. Specifically, the encoder measures the time taken by the motor to traverse two teeth and factors in the total number of teeth, which, in this case, corresponds to 468 holes in the encoder wheel.

2.2 Initial setup and legacy code review

To begin our participation in this project, our group conducted a study in the materials delivered by the group that treated this subject during last semester. The following section identifies key areas that our group identified, along with recommendations from the previous group on what could be improved in this project.

2.2.1 Initial state of the project

At the start of the project, we received a robot with good physical stability and no major electronic issues. The robot was capable of moving in a straight line and making approximate turns. The code was already compartmentalized, making it easier to identify and work on specific functionalities. While serial communication between the Raspberry Pi and the Arduino card was already established, and the ultrasonic sensor detection system was well-implemented (successfully sending obstacle data to the Arduino), several issues remained.

A closed-loop system for motor speed control was implemented using the encoders ; however, this system was not entirely optimal. The times-step calculus method was unreliable at low speed, while being quite efficient for higher speed. This led to a lack of precision with low angles of rotation.

Additionally, an obstacle avoidance algorithm was in place but was overly simplistic and unreliable. When an obstacle was detected within a certain range, the program triggered a pre-defined sequence of movements, assuming the obstacle was a standard cube. This approach relied on arbitrary parameters, such as a fixed turning time of two seconds, which were unrelated to real-time obstacle detection. Furthermore, the motor power was directly linked to the battery's charge level, which varied and affected performance inconsistently. As a result, the robot's turning behavior was erratic, and the obstacle avoidance algorithm was practically ineffective. A SLAM algorithm could be used instead, to improve the current obstacle avoidance method.

2.2.2 Project's objectives and Minimal Viable Product

Building on the idea of implementing a SLAM algorithm, one of the key goals of this project is to integrate a LiDAR system — both physically, by mounting the sensor onto the robot's frame, and algorithmically, by developing SLAM-based code capable of mapping the environment and enabling more intelligent navigation.

In parallel, we aim to significantly improve the speed control of the omnidirectional wheels. To ensure more accurate movement in space, we will move away from a simple PID controller and instead implement a nonlinear control strategy based on a motor model that includes friction.

Lastly, we plan to add current sensing capabilities to measure the real-time average current in the motors. This information will be essential to dynamically refine the control system based on the actual effort required by each motor.

Our **Minimal Viable Product (MVP)** is a robot capable of navigating and avoiding a fixed rectangular obstacle — thus maintaining or improving upon the performance level achieved by the previous group.

One of our main success criteria will be to obtain a greater turning radius when moving in a straight line, indicating improved speed regulation and stability compared to previous implementations.

If time permits, we will also explore the possibility of enabling the robot to follow a precise circular trajectory, which would further demonstrate the effectiveness of our motion control improvements.

Chapitre 3

Speed Control of Wheel Rotation

This section is dedicated to the speed control of the wheel rotation. Ensuring a precise rotational speed of the wheels is a key element in enabling accurate movement of the robot in space, as the relative speeds of the omnidirectional wheels determine the robot's overall direction of motion. It is therefore necessary to implement an initial control loop at the wheel level.

We thus focus on a speed control strategy managed through PWM current modulation. Previous groups implemented a control system using a PID controller but encountered several non-linearity issues : notably a dead zone between -30% and 30% duty cycle, hysteresis (explained by the difference between static and dynamic friction coefficients), and so on.

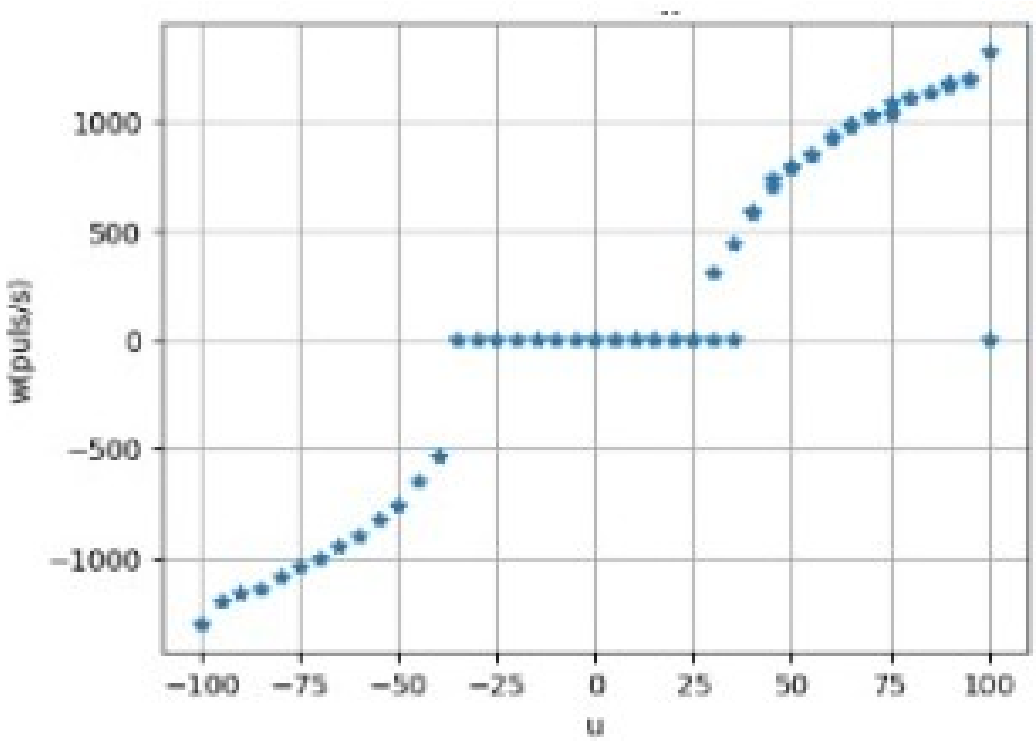


FIGURE 3.1 – Rotation Speed as a function of the Voltage's percentage

Moreover, since most tests were carried out with the robot elevated, we can expect additional perturbations during real-world testing (such as the robot's inertia, the effort deployed by the other 3 wheels, etc.).

The limited effectiveness of the previous years' results now leads us to consider a nonlinear controller and adopt the architecture illustrated in the following diagram.

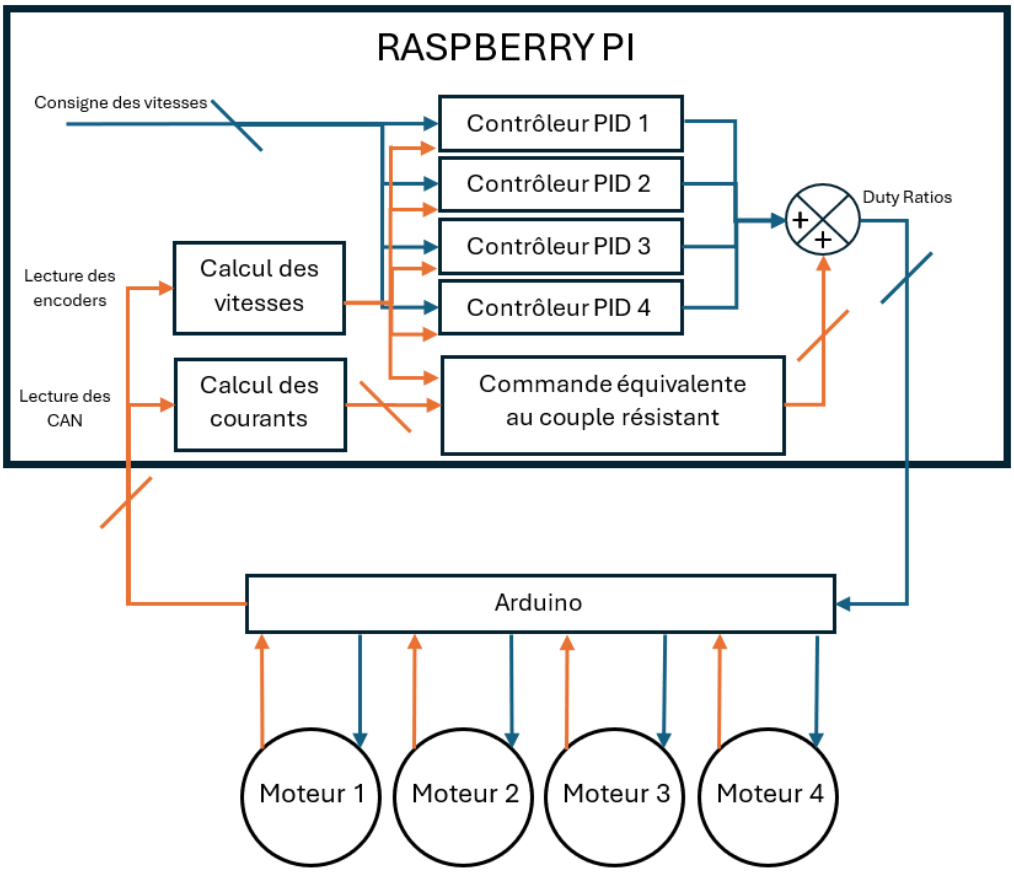


FIGURE 3.2 – Simplified architecture of the nonlinear speed control loop.

The calculations are performed with the Raspberry because it is what measures the real time position of the wheels via the incremental encoders, which imposes an exchange with it.

Chapitre 4

Current Probe and Electrical Analysis of the Motor

4.1 Motivation

Motor control is inherently challenging and imprecise in both speed and position due to significant non-linearities. The primary source of these non-linearities stems from dry friction, which is difficult to compensate for due to its variability. Friction effects depend on speed, wheel angle (tests with elevated wheels have shown inconsistent speeds under constant voltage, indicating a slight imbalance), robot weight, and, most importantly, the inertia of the robot and disturbances from other wheels during turns. Accurate servo control is most critical in these cases, as precise angular positioning is necessary (either by velocity control for circular arcs or position control for pivot turns).

The core issue can be summarized as follows : imprecise motor servoing due to unpredictable, highly non-linear parasitic torques (image below). Our goal is to develop a method to counteract these disturbances. The effectiveness of this correction will be assessed by comparing servo precision—both in speed and position—for various setpoints.

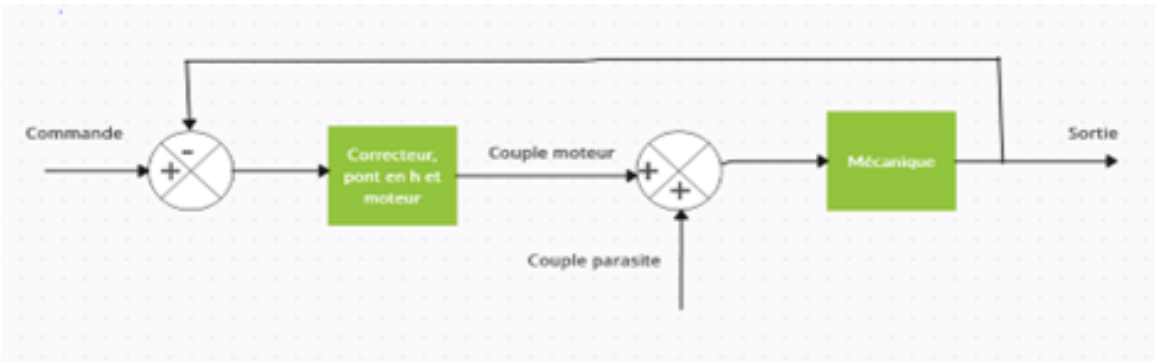


FIGURE 4.1 – Nonlinear parasitic torque affecting motor control.

A common solution for nonlinear control is compensation—adding a torque equal and opposite to the parasitic torque C_p after the controller to restore a more linear motor behavior. This allows the system to operate within conditions where classical controllers (such as the PID used by the previous group) are effective.

Problem : The torque is not constant/simple. Therefore, we want to be able to calculate it (approximately) in real-time. To do this, we can compare the acceleration of the wheel $\frac{d\omega}{dt}$ and the motor torque C_m :

We therefore need to : measure the inertia J, then the acceleration $\frac{d\omega}{dt}$ in real time using the incremental encoders, and finally the motor torque C_m through a current measurement and an electrical model of the motor.

4.2 Measurement of Angular Acceleration ($\frac{d\omega}{dt}$)

To compute the angular acceleration $\frac{d\omega}{dt}$ using incremental encoders, it is first necessary to ensure the reliability of the velocity data they provide. Since encoder signals correspond to position, obtaining acceleration requires performing two successive differentiations. However, it is well established that numerical differentiation significantly amplifies signal noise. Consequently, filtering of the encoder outputs becomes essential. This filtering

- currently implemented as a moving average, though alternative low-pass filters could be employed - introduces a delay in the feedback signal. In control systems, such delays within feedback loops are known to be potential sources of instability. It is therefore crucial to compare the characteristic time of the wheel speed dynamics with the sensor feedback delay. Our objective is to ensure that the system's dynamic response is substantially slower than the sensor update rate. Specifically, we aim for a ratio of at least 10 between these two characteristic times to guarantee stable and reliable control performance.

4.2.1 Characteristic time of Wheel Speed Dynamics

To estimate the characteristic time associated with wheel speed variations, we rely on data obtained from an open-loop experiment conducted by a previous project group.

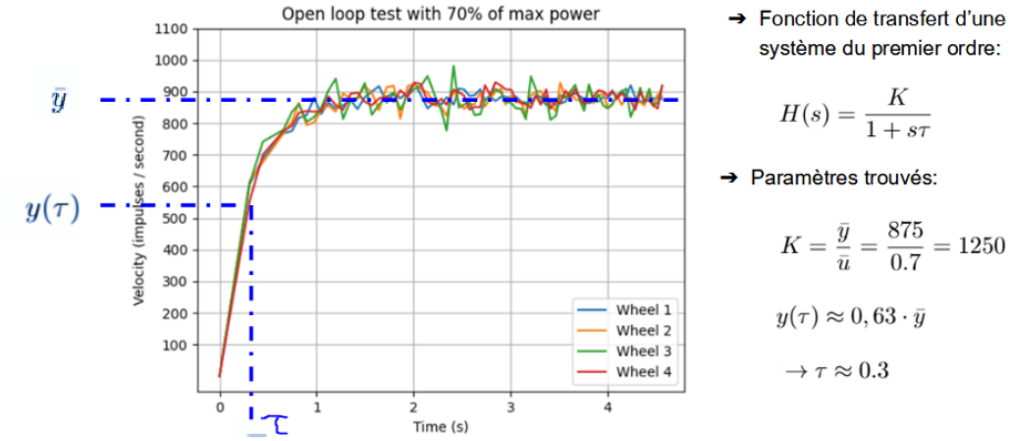


FIGURE 4.2 – Analysis of the Characteristic Time of Wheel Speed Evolution

From the test, we obtain a characteristic time on the order of 0.3 seconds. This experiment was conducted with the robot lifted off the ground, such that the wheels were not in contact with any surface. As a result, the robot's inertia was not a factor, whereas in normal operation, it would reduce the acceleration. This 0.3-second duration should therefore be considered a lower bound, representing the best-case scenario for the system's dynamic response time.

4.2.2 Characteristic Time of Sensor Feedback

A low-pass filter applied to discretised signals generates output based on a history of previous input values. The characteristic time in this context corresponds to the interval between the first and last input values that influence the current output. We initially applied a moving average filter, and it was necessary to use a window of approximately 20 seconds to obtain a sufficiently smooth velocity signal. To estimate acceleration from this filtered velocity, an additional averaging over the same time span was required. This results in a worst-case ratio of approximately 15 between the system's dynamic response time and the sensor feedback delay. Although this ratio satisfies our initial target of 10, it is still relatively close to the lower limit. Therefore, we aim to maximize this ratio to improve control robustness.

4.2.3 Reducing the Characteristic Time of Sensor Feedback

We use a quadrature (two-channel) encoder. Previous implementations utilized one channel to detect changes in wheel position and the second solely to determine direction. However, both channels can be used to detect transitions, effectively halving the interval between pulses and thereby doubling the measurement resolution. Below is a snippet from the previous version of the encoder logic in encoder.py :

```
clkState1 = GPIO.input(clk1)            # The program observes both channels
dtState1 = GPIO.input(dt1)

if clkState1 != clkLastState1:          # If a change is observed in the first channel
    if dtState1 != clkState1:           # If the two channels are returning opposing signals
        counter1 += 1                   # Then the wheel is rotating in the positive direction
        dir1 = 1
        t1 = time.time()
    else:
        counter1 -= 1                   # Else the wheel is rotating in the negative direction
        dir1 = -1
        t1 = time.time()

clkLastState1 = clkState1
```

Below is a snippet from the new version of the encoder logic in encoder.py :

```
while True:

    clkState1 = GPIO.input(clk1)      #The program observes both channels
    dtState1 = GPIO.input(dt1)

    if clkState1 != clkLastState1:    #If a changed is observed in the first channel
        if dtState1 != clkState1:     #If the two channels are returning opposing signals
            counter1 += 1             #Then the wheel is rotating in the positive direction
            dir1 = 1
             t1=time.time()
        else:
            counter1 -= 1         #Else the wheel is rotating in the negative direction
            dir1 = -1
            t1 = time.time()
    if dtState1 != dtLastState1:      #If a change is observed in the second channel
        if dtState1 == clkState1:     #If the two channels are returning identical signals
            counter1 += 1             #Then the wheel is rotating in the positive direction
            dir1 = 1
            t1=time.time()
        else:
            counter1 -= 1             #Else the wheels are moving in the opposite directions
            dir1 = -1
            t1 = time.time()

    dtLastState1 = dtState1
    clkLastState1 = clkState1
```

This modification proved to be time-consuming, as it required navigating through non commented code, with no clear indication of which functions handled encoder readings or which variables corresponded to which inputs. Nevertheless, by reducing the time between two encoder input pulses, it became possible to average over time windows that are twice as short, thereby improving the responsiveness of the feedback loop.

As a result, we obtained a worst-case ratio of 30 between the characteristic time of wheel speed dynamics and the sensor feedback delay, which is a much more satisfactory margin.

In addition, we performed a comparison between the wheel rotation angle reported by the encoders and the true rotation angle, obtained through chronophotographic analysis, in order to verify that the theoretical resolution was indeed achieved and that the Raspberry Pi was not missing any encoder pulses. The measured ratio of 937.2 pulses per wheel rotation (calculated as : number of slots per channel $\times$ 2 $\times$ gear reduction ratio $\times$ 2) confirmed the high precision of our motor measurement setup.

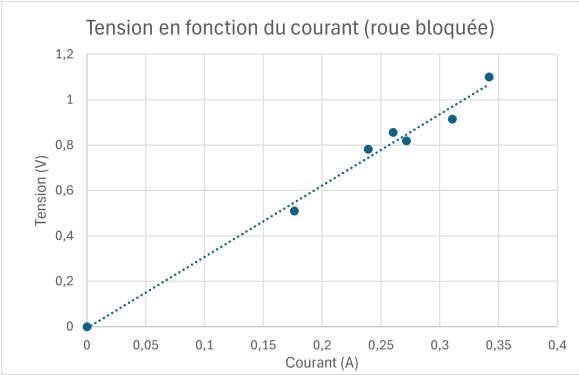
4.3 Motor Analysis

Unlike previous groups, we now examine the motor from an electrical perspective to determine its resistance R and coupling coefficient K , as these parameters were absent from the motor datasheets.

To measure R , the motor was locked to eliminate back electromotive force (EMF). The values were recorded using a Waveforms oscilloscope, with current measured via a shunt resistor. Given that the control is PWM-based, voltage U , current I , and rotation speed ω were averaged over long periods (5 seconds), significantly

exceeding the PWM cycle (1ms). Measurements were conducted on a single motor for now ; additional tests on the remaining three motors will follow once the measurement system is fully validated.

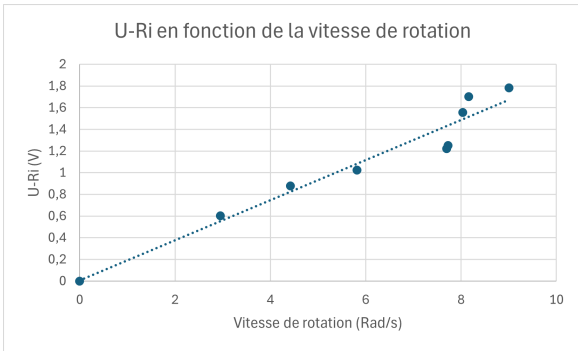
Measured values :



Motor blocked

Resistance : $R = 3.11\,\Omega$
Coefficient of Determination : 0.997

(a)



Motor unblocked : U-Ri = Kω

Torque Coefficient : $K = 0.19\,\mathrm{V\,s}$
Coefficient of Determination : 0.989

(b)

FIGURE 4.3 – Electrical characterization of the motor in blocked and unblocked configurations.

We do obtain results that appear linear, and we neglect the temperature dependence of R and K, as the previous group demonstrated that the motor behavior varies little.

4.4 Signal Characteristics

Below is a waveform showing motor current (orange) and voltage (blue) over time, measured using a Waveforms oscilloscope and a $0.5\,\Omega$ shunt resistor :

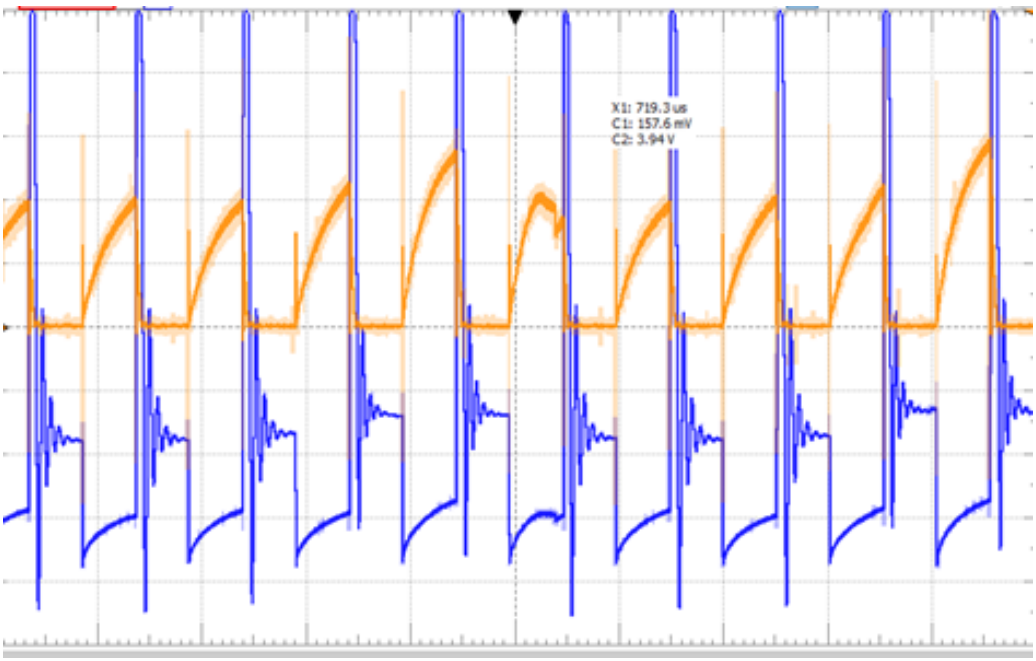


FIGURE 4.4 – Current (orange) and voltage (blue) measured across the motor using an oscilloscope.

We notice :

• The inductance of the motors is not negligible, and the current takes time to build up. This contributes to the non-linearity of the motor's response to the PWM control, observed by the previous group. Indeed, the current, and thus the torque, is not at its maximum throughout the duration when the H-bridge is conducting. The integral of the torque is therefore not proportional to the proportion of the time during which the H-bridge

is conducting, and thus to the PWM value. Because we want to impose a specific torque on the motor (cf figure 3.1) we should consider controlling it through current regulation rather than PWM. This can be achieved either by current feedback control or by using calculations to convert current to PWM if the electromotive force (EMF) is known. It is possible to calculate that force by multiplying the speed of the motor shaft by the already determined torque constant.

• The voltage across the motor terminals is far from a true square wave signal. This is due to the transistors in the H-bridge, which exhibit an LC behavior, noticeable even at 1 kHz.

4.5 Current Probe Implementation

It would therefore be difficult to calculate the current analytically due to its complex shape. Let us recall that the goal is to measure it, or at least to measure its average value over a PWM period in order to estimate the motor torque. We therefore propose to measure it in real time using a current probe. We want to use a shunt resistor, connected to the analog pins of the Arduino, as it is the least expensive option, compared to using a hall effect sensor. However their sampling frequency are 9.6kHz by default, not enough for measuring the average value of a 1kHz signal (the frequency of the PWM). The sampling frequency can be increased, by sacrificing the precision of one sample. When the electric montage will be completed and connected to the Arduino's pins, a study of the best sampling frequency to maximize the precision of the measurement of the average value of the current will have to be done. The differential voltage across the shunt resistor must also be transformed into a voltage readable by the Arduino, meaning a voltage relative to the Arduino ground, which must not become negative or exceed 5V.

We propose using an instrumentation amplifier (INA118P) to convert the differential voltage into a relative voltage, and to add a 2.5 V offset to account for the fact that the current can flow in both directions, and thus the voltage can change direction. To obtain this offset using only the Arduino's GND and 5 V supply, the V- pin of the amplifier is connected to the Arduino ground, the V+ pin to 5 V, and the reference is connected to a virtual ground at 2.5 V created through a voltage divider (diagram below). The voltage divider consists of large resistors (20 kΩ) to avoid drawing too much current from the battery, but this is only suitable if the amplifier is directly connected to the Arduino pin, which draws very little current. If a subsequent stage wants to add an analog filter that draws more current, the resistors will need to be resized to ensure the amplifier can draw all the current it requires. The gain is set by adjusting the resistance between the RG pins of the amplifier, via $G = 1 + \frac{50\,\mathrm{k\Omega}}{R_G}$. After measuring the current at maximum power, we have chosen a gain of 3 to stay within the range of the Arduino pin.

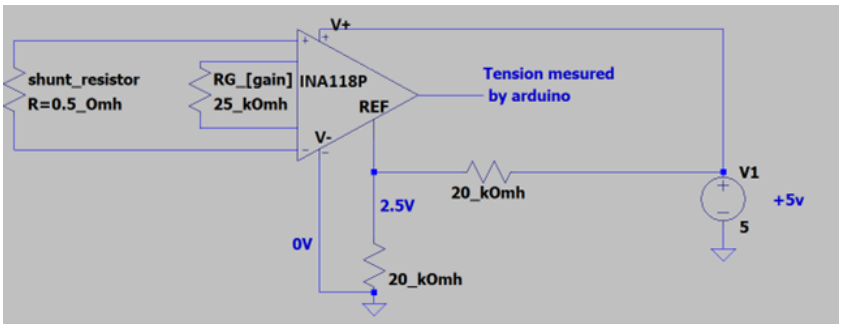


FIGURE 4.5 – Wiring diagram of the differential amplifier with virtual ground offset.

4.6 Power Supply Issues

The battery connected to the motors of the wheels seems to have a defect and is not delivering a constant voltage. A periodic pattern at 50 Hz can be observed (in red below).

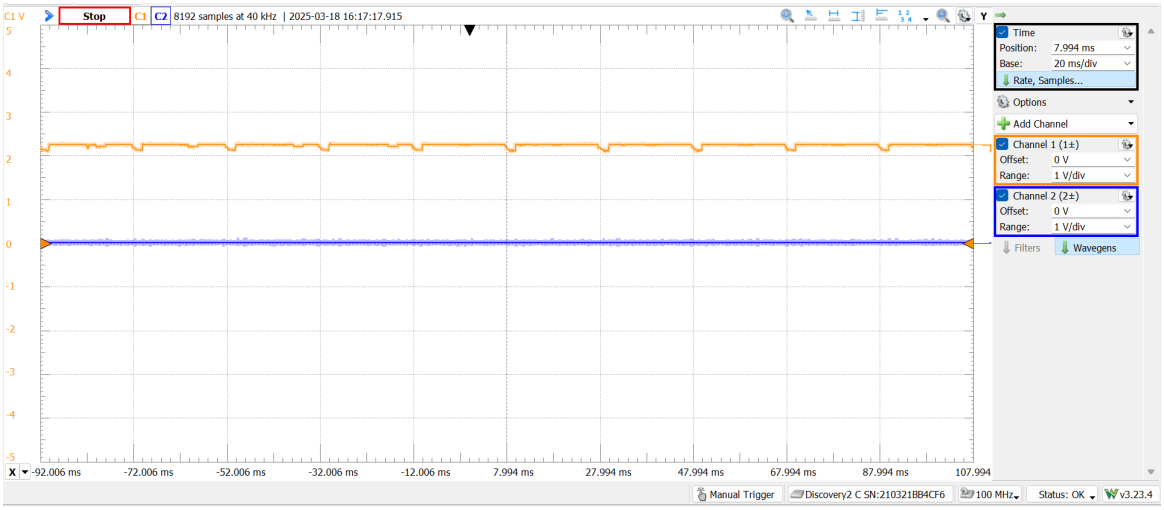


FIGURE 4.6 – Unstable battery signal showing a 50 Hz periodic defect.

Since this corresponds to the frequency of the electrical grid, we wondered if it was really a defect with the battery or if it could be a fault with the measurement device, as it is connected to mains power, or possibly from the wires acting as antennas. However, we did not observe this pattern in other measurements, so we assume it is indeed a battery defect, potentially due to the fact that the battery is stabilized, which suggests the use of active components tuned to a grid frequency to allow for mains recharging.

This defect seems for now minor as it only account for about ten percent of the tension delivered by the battery. However, it can be the explanation if an unexpected signal is noticed at 50 Hz, as it affects the gain and offset of the amplifier.

4.7 Signal Conditioning Prior to the Instrumentation Amplifier

According to the INA118P datasheet, the amplifier is expected to reproduce an output signal closely matching the input signal up to frequencies of approximately 250 kHz. However, it is necessary to filter the input signal, as instrumentation amplifier are well known for significantly amplifying high-frequency noise.

The following result illustrates the effect of passing the input signal - namely, the voltage across the shunt resistor, which reflects the current through the motor - through an RC low-pass filter with a cutoff frequency of 100 kHz.

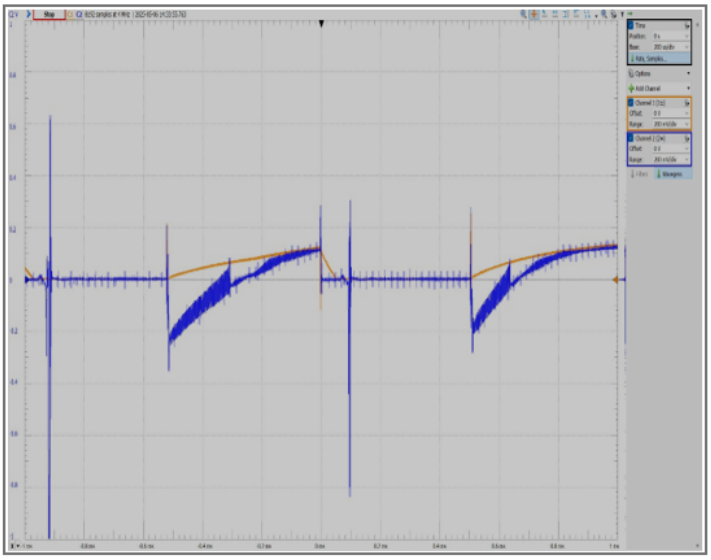


FIGURE 4.7 – Input and Output Voltages of the Instrumentation Amplifier, Measured with Waveform Oscilloscope (in Volts)

Orange : Input voltage (voltage across the shunt resistor, filtered at 100 kHz)
Blue : Output voltage of the amplifier

It appears that the amplifier is unable to accurately follow the input signal : the expected waveform is not preserved, and the output exhibits noticeable spikes and noise artifacts. We hypothesized that this behavior might be due to power supply instability, potentially caused by current draw exceeding what the supply can provide. To investigate this, we measured the voltage across all of the amplifier's power supply pins while a signal was applied to its input, in order to verify whether the 5V supply from the Arduino remained stable under load, which it did. It then means that the amplifier is amplifying high frequency noise, even though this noise has been reduced by the filter. The following result corresponds to a setup in which the input signal is filtered by an RC low-pass filter with a cutoff frequency of 10kHz.

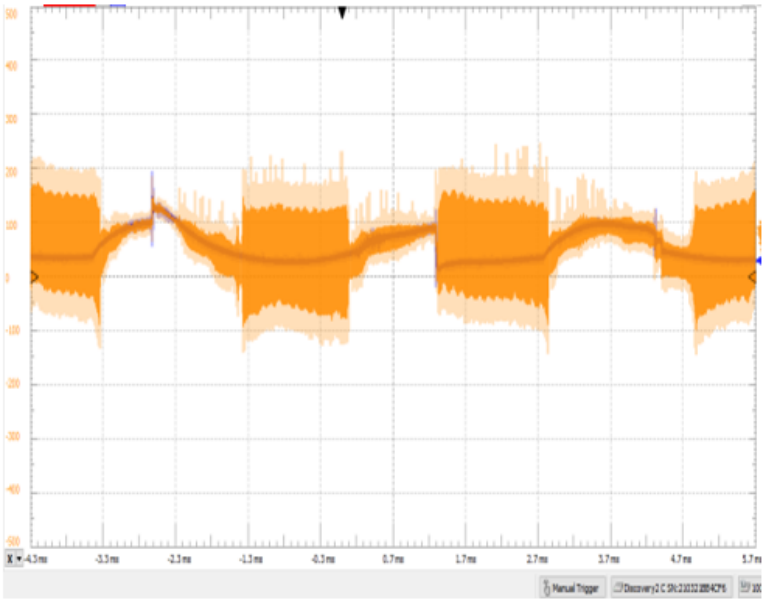


FIGURE 4.8 – Input and Output Voltages of the Instrumentation Amplifier, Measured with Waveform Oscilloscope (in mV)

Orange : Output voltage of the amplifier
Dark trace, centered within the orange oscillations : Input voltage (voltage across the shunt resistor, filtered at 10 kHz

It can be observed that the amplifier is now capable of following the input signal—as the output oscillates around the input—a low-pass filtering of the output would be required to recover the original input waveform. However, it is also clear that the input signal itself has lost its original shape. This leads to the following conclusion :

• If the input signal is insufficiently filtered, the amplifier fails to operate correctly, as it amplifies high-frequency components, even though these are partially attenuated by the RC filter.

• Conversely, if the signal is sufficiently filtered for the amplifier to function properly, it loses important features of its shape. This degradation may be explained by the presence of discontinuities or rapid transitions in the input signal, which require high-frequency components (well above 1kHz) to be accurately represented. When those components are removed through filtering, the signal becomes distorted and no longer reflects the true motor current dynamics.

It is then impossible to obtain a relative tension accurately depicting the differential tension across the resistor using only an INA118P amplifier and a RC filter.

4.8 Remaining Tasks

Thus, to complete the implementation of the control system, it remains necessary to address the current sensor issues. This could involve using a higher-order filter—as only a simple RC filter was tested due to time constraints. A more sophisticated filter could more effectively attenuate high-frequency components while maintaining a sufficiently high cutoff frequency to avoid distorting the input signal. Alternatively, a Hall effect sensor could be employed, as it directly produces a signal proportional to the current.

Once an appropriate sensor is selected, its output must be connected to the Arduino. Calibration will also be required, since inaccuracies in the resistor values (as shown in Figure 4.5) can result in offset and gain errors. The overall accuracy of the current sensing system must then be experimentally verified. In addition, the moment of inertia J still needs to be measured.

Finally, the control parameters must be chosen. These include not only the PID controller coefficients, but also the analog input sampling frequency of the Arduino (as discussed in section 4.5), and the averaging window applied to the signal from the incremental encoders. These last two parameters affect the responsiveness of the sensors, but at the expense of measurement accuracy.

An empirical study, based on a large number of experiments, can be conducted to identify the optimal values of these parameters in order to maximize the key performance indicators—which are, as a reminder : the

accuracy of the angular position response when a precise rotation is commanded, and the accuracy of the speed response under a velocity command.

Chapitre 5

Obstacle Detection

The success of autonomous navigation relies largely on accurate perception of the environment, achieved through the use of various sensors. In the previous group's robot version, an ultrasonic sensor was used for obstacle detection, allowing only the presence (or absence) of an obstacle directly in front of the robot to be detected. Although functional, this ultrasonic sensor had certain limitations in terms of accuracy, range, and response time, particularly in dynamic or complex environments. In this chapter, we will implement LiDAR (Light Detection and Ranging) technology as a complementary solution to improve navigation efficiency and obstacle avoidance capabilities.

5.1 Initial Sensor Layout

The initial layout relied solely on an ultrasonic sensor for obstacle detection. This sensor emits sound waves and measures the time of flight (ToF) of the reflected signal, a method that allows for reasonable obstacle detection, but has some limitations :

- Limited accuracy : The sensor provides approximate distance measurements, often with notable noise and errors, especially when detecting small or angled objects. A conical detection area can cause false positives when objects off the intended path reflect the sound waves.
- Slower response time : The ultrasonic sensor operates sequentially, requiring a pause to transmit and receive signals, which can delay real-time detection and trajectory adjustments.
- Sensitivity to surface types : The sensor performs poorly on absorbent surfaces (e.g., soft tissue) or irregularly shaped objects, as the sound waves may not be reflected effectively.
- "Binary" implementation : The code architecture at the beginning of the project used a single measurement from the robot to check whether the distance between the sensor and the obstacle was below a certain threshold. The only data used was an indicator indicating the presence or absence of an object.

These limitations prompted the previous group to research additional or alternative sensing technologies to improve the robot's perception, particularly in scenarios requiring accurate and rapid obstacle detection. Infrared sensors, a LiDAR, and activation of the Raspberry Pi camera module were discussed by the previous group.

5.2 LiDAR Decision

LiDAR (Light Detection and Ranging) sensors emit laser beams and measure the return time of the reflected signal. This technology provides highly accurate distance measurements and generates detailed environmental maps, making it ideal for advanced robotics applications. With high accuracy and range, LiDAR sensors offer centimeter-level accuracy and can detect objects at distances greater than 10 meters, far surpassing ultrasonic and IR sensors. Furthermore, the LiDAR we have is capable of performing a 360-degree scan, allowing the robot to detect obstacles and comprehensively map its environment. Fast and more versatile in the face of surface and environmental constraints, LiDAR is an excellent way to improve autonomous navigation.

Figure 5.1 – Image d'un Rplidar C1

Integrating the LiDAR into the system required an implementation in two distinct dimensions : physical and software. These two dimensions were addressed separately.

5.3 Physical and Electronic Integration

The main discussion regarding the physical dimension focused on the position of the LiDAR. Since the "RPLiDAR C1" LiDAR model was now capable of detecting only a 2D plane parallel to the ground, its correct positioning became crucial. It could only detect obstacles located at a certain height. Its 360° range made it inaccessible if placed anywhere other than at the top of the robot. However, its position at the top of the robot only allowed it to detect obstacles of a height equal to its own. After careful consideration, given the complexity of the robot's reorganization and our main objective for adding sensors : to obtain more accurate data about its environment, we decided to install it at the top of the robot, assuming that all the obstacles to be overcome would be high enough to be detected. Thus, to install the LiDAR at the top of the robot, we added an additional layer, increasing the robot's height without increasing its weight.

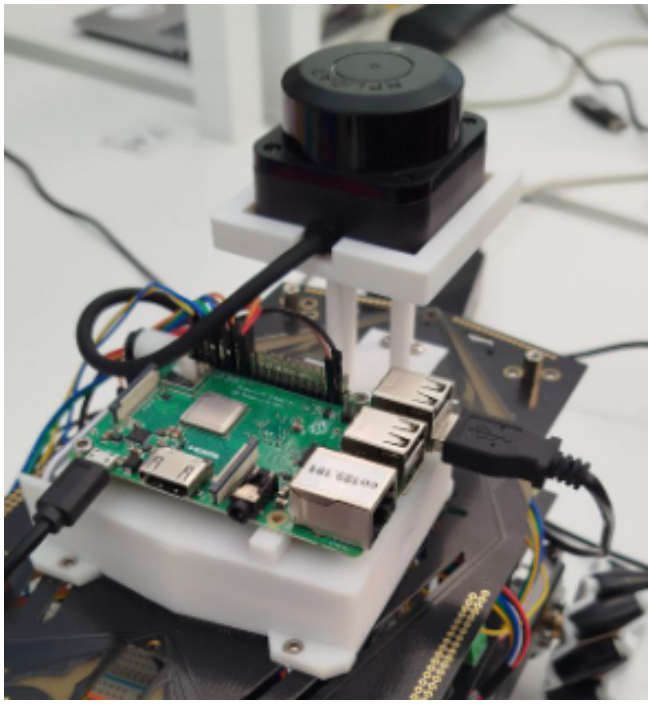


Figure 5.2 – Support du Lidar

The LiDAR and its mount are positioned 20 cm above the ground, so obstacles must have a minimum height of 20 cm to be detected.

Since the robot is a part we share with the entire team, we decided to conduct all LiDAR testing on another Raspberry Pi identical to the one used by the robot. The next step is to connect the robot and the Lidar.

5.4 Data Acquisition (Software integration)

The LiDAR sensor is provided with a C++ library for data extraction and processing. To bridge this gap, we reused the communication system based on sockets set up by the previous group. Sockets allow two programs to communicate over a network by establishing a connection through specific ports. In our case, the C++ program responsible for managing the LiDAR sensor sends the data to the control software written in Python via this communication channel, thus facilitating data exchange despite the difference in programming languages.

This has been a long process because the library needed to compile the LiDAR was outdated. At the current stage, we are able to activate the LiDAR's motor and retrieve the data it sends. The data is collected in the form of an array containing elements in the form of (angle, distance), where the angle is expressed in degrees and the distance in millimeters.

1022	173.87	815.0
1023	174.57	821.0
1024	175.1	0.0
1025	176.0	2682.0
1026	176.7	2695.0
1027	177.23	0.0
1028	178.12	132.0
1029	178.64	0.0
1030	179.36	0.0
1031	180.07	0.0
1032	180.95	774.0
1033	181.65	773.0
1034	182.19	0.0
1035	0.59	193.0
1036	1.3	194.0
1037	2.0	196.0
1038	2.7	197.0
1039	3.41	197.0
1040	4.11	197.0
1041	4.81	197.0
1042	5.52	197.0

FIGURE 5.3 – (angle,distance)

The LiDAR rotates clockwise and makes steps of approximately 0.70 degrees. The 0-degree angle is marked by a reference on the LiDAR. When the LiDAR reaches 360 degrees, it returns to 0 degrees to start a new sampling cycle. Another issue we encountered is that we observe the LiDAR starting from 0 degrees and generally reaching an angle between 180 and 230 degrees before returning to 0 degrees. Sometimes, it manages to send measurements with angles up to 360 degrees, allowing for a complete scan of the environment, but this happens rarely—perhaps once every twenty sampling cycles. What the previous group did was continue rotating the LiDAR until it obtained a full sequence. One of our current hypotheses is that the sampling frequency is too high, which would explain why only a portion of the packets is sent most of the time.

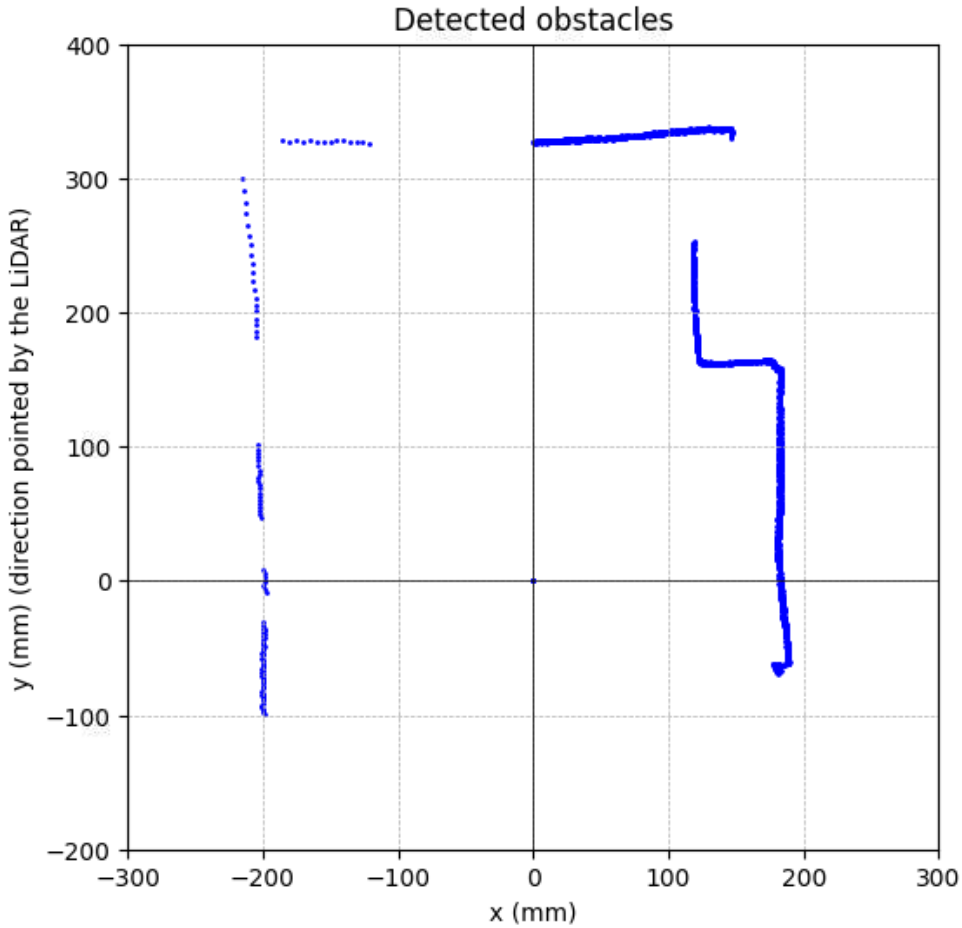


FIGURE 5.4 – Environment scan before program optimization. The robot is at point $(0, 0)$, facing the line $x = 0$.

In the figures above, we placed three walls around the LiDAR, using computer cases to form the walls : one in front, one to its left, and one to its right. We also placed the LiDAR's casing in this environment to have an additional reference point. We observe a significant lack of measurements between 180 and 360 degrees (in the clockwise direction), as previously explained.

The issue actually stems from the fact that with each data acquisition, the program opens a document to record the data. Since the operating system's authorization is not immediate, some pending data is not processed and is deleted to make room for new data. After optimizing the code, we can finally appreciate below the expected result after processing data from a functional LiDAR.

FIGURE 5.5 – LiDAR environment

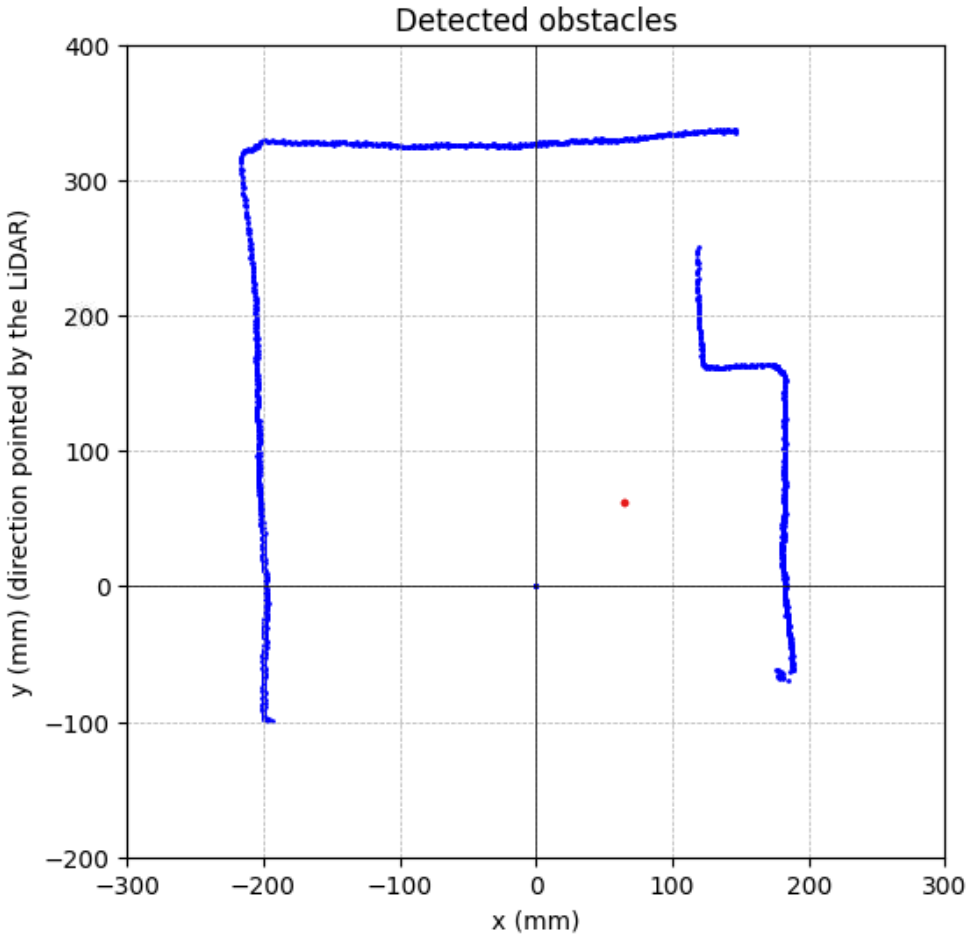


FIGURE 5.6 – Environment scan after program optimization. The robot is at point $(0, 0)$, facing the line $x = 0$.

We can note the presence of cardboard sheets placed around the computer cases. Without them, the LiDAR detects virtual points caused solely by parasitic reflections of the emitted waves on the metal casings of the computers. This results in inaccurate obstacle detection. In the figure below, we scanned the same environment as before with our functional LiDAR, except that we did not include the LiDAR's box or the cardboard sheets.

FIGURE 5.7 – LiDAR environment with outlier points

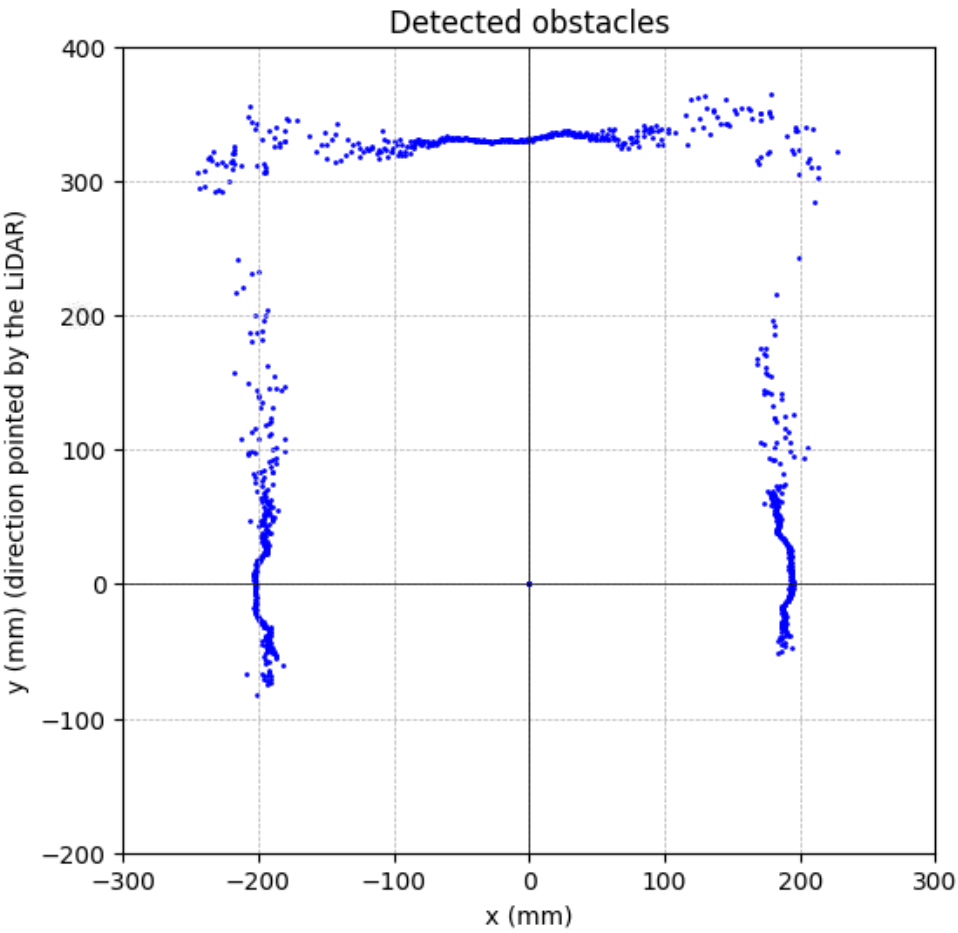


FIGURE 5.8 – Parasitic reflections. The robot is at point $(0, 0)$, facing the line $x = 0$.

5.5 Landmark Detection using RANSAC

Once the data has been correctly acquired, the next step was landmark detection. When a feedback loop is established on the robot, we achieve a certain precision on each of its movements. However, after many

iterations, it becomes impossible to know the robot's exact position. For example, if the robot can rotate 90 degrees with an accuracy of 0.1 degrees, after four million such rotations, it will clearly no longer be pointing in the original direction. The goal is therefore to map the environment and locate the robot within it.

For this, we based our work on the previous group's results.

The RANSAC (Random Sample Consensus) algorithm is an iterative method used to identify and fit models to data while filtering out outliers. It works by randomly selecting a small subset of data points to estimate a model, such as a line in 2D space. The algorithm then checks how many of the remaining points fit this model within a given tolerance. If enough points agree, the model is refined using all inliers (points that fit well) with a least squares method. This process repeats multiple times, each time removing the points belonging to the detected lines. RANSAC is particularly useful for detecting structures in noisy data. In this case, for example, it allows the robot to ignore people and moving objects around it.

This algorithm depends on three key parameters :

— Distance Threshold : The maximum distance a point can be from the estimated line to be considered an inlier. A lower threshold results in a stricter fit, while a higher threshold allows more noise tolerance.

— Inliers Ratio : The minimum proportion of points that must fit the model for it to be considered valid. A higher ratio ensures the model represents the majority of the data.

— Number of Iterations : The number of times the algorithm randomly samples points to find the best model. More iterations increase the chance of finding the optimal solution but require more computation time.

Changing the distance threshold and inliers ratio has a similar impact : if set too high, precision decreases ; if too low, no consistent lines are detected.

Below is an example of the RANSAC algorithm applied to our LiDAR data :

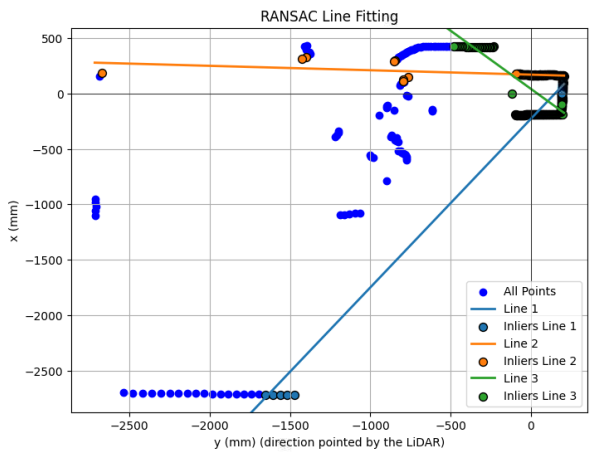


FIGURE 5.9 – Landmark Detection with RANSAC. The robot is at point $(0, 0)$, facing the line $x = 0$.

We slightly modified this approach to better suit our specific needs.

However, compared to the convex hull method we developed afterward, RANSAC presents several drawbacks :

— Fragmentation : RANSAC detects independent line segments, which can fragment a continuous obstacle into multiple parts, complicating a global obstacle representation.

— Parameter Sensitivity : Detection quality strongly depends on parameter tuning ; poor settings can cause missed or incorrect line detections.

— Computational Cost : RANSAC may require many iterations and distance computations, especially with dense point clouds, increasing runtime.

— Limited Compact Representation : RANSAC outputs multiple line equations but does not directly provide a compact geometric shape of obstacles, unlike convex hulls which summarize clusters as polygons.

— Less Robust for Complex Shapes : Convex hulls naturally approximate the shape of point clusters, capturing more geometric information, whereas RANSAC is limited to linear segments.

These limitations led us to develop the convex hull-based method, which provides a more structured and robust obstacle representation for navigation and localization.

5.6 Landmark detection using convex hull generation

To generate a structured and exploitable map of the environment, we developed an obstacle detection algorithm based on the construction of convex hulls. The underlying idea is simple : raw LiDAR data, initially

expressed in polar coordinates, is first converted into Cartesian coordinates to visualize the resulting point cloud in the plane. From this cloud, a clustering algorithm (here, `DBSCAN`) is used to identify local groupings of points, which can be interpreted as physical obstacles in the environment.

Once the clusters are extracted, a convex hull is computed for each one—this is the smallest convex polygon that encloses all the points in the cluster. This geometric envelope gives each obstacle a compact and coherent shape. The centroid (or geometric center) of the convex hull can then serve as a reference point for analyzing the robot's movement (rotation or translation), while offering robustness to noise and small sensor shifts.

The algorithm relies on several tunable parameters :

- `eps` in DBSCAN, which defines the maximum distance between two points to be considered neighbors. A smaller value detects more compact obstacles, while a larger one aggregates broader structures.
- `min_samples`, the minimum number of points required to form a cluster. This helps to filter out noise (e.g., sparse and insignificant point groups).
- The duration of LiDAR data acquisition (here set to 3 seconds), which directly affects the density of the point cloud and, therefore, the mapping quality.
- The maximum scanning distance, which limits detection to nearby obstacles, more relevant for immediate navigation.

This method not only allows effective visualization of obstacles but also produces structured representations that can be reused for navigation, localization, or motion analysis. It offers a useful geometric approximation of the actual environment, especially for short-range perception.

In addition, this approach also served as a foundation for estimating the robot's rotation. By comparing the positions of centroids between two acquisition steps, it becomes possible to calculate the angle of rotation undergone by the robot or the LiDAR system (see next section).

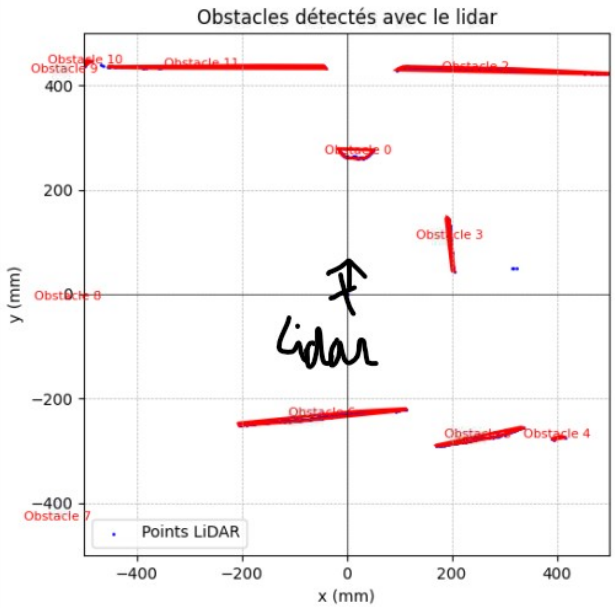


FIGURE 5.10 – Detection in his environment using convex hull generation. The lidar is at point $(0, 0)$, facing the line $x = 0$.

FIGURE 5.11 – Lidar/robot environment

This figure illustrates that the algorithm generates polygons corresponding to the visible parts of the obstacles. For example, you can observe a half-cylinder shape and several gaps at the bottom of the figure. This raises certain challenges, as we will discuss later, because when the robot translates to the right, new portions of the obstacle at the bottom become visible, indicating that the robot cannot pass through.

However, this mapping only captures the visible surfaces of obstacles (you can see only , that is, those facing the sensor at the moment of measurement. Hidden or rear-facing areas are naturally excluded. Finally, this algorithm was developed during the final stages of the project, and we did not yet have the opportunity to fully integrate it into the navigation loop or to rigorously assess its accuracy and robustness.

5.7 Controlled Movement : Translation and Rotation

In this section, we present the robot's controlled movement : advancing a set distance followed by a rotation by a given angle. These basic motions are key to navigation. We also cover how the robot's pose is estimated and refined using sensor data to account for errors.

5.7.1 Forward + distance

Once we had implemented the lidar from a physical and software perspective, we created an initial algorithm to advance a given distance.

How does it work ?

Let's imagine we want to move forward by a given distance d.

1. The lidar measures the distance to the obstacle in front of the robot.
2. If the measured distance is greater than d, the robot moves forward by a sufficient distance d' during time t.
3. We measure the remaining distance left to travel.
4. The robot moves forward for a short time t'.
5. We measure the remaining distance again.
6. We repeat the last two steps until the robot has moved at least the desired distance d.

The question remains : how to quantify "sufficient distance" and "short time" ? In theory, we take :

$$t = \frac{d}{v_{\text{average robot}} + \epsilon}$$

with $\epsilon \to 0$ and $t' \to 0$.

The term ϵ is used to slightly overestimate the robot's average speed to ensure that the robot never overshoots d on the first move. This allows the robot to converge toward d, and if t' is infinitesimally small, it will only exceed it by an infinitesimally small amount.

The smaller ϵ is, the closer d' will be to d from below, and the smaller t' is, the closer the final position will approach d from above.

However, the smaller t' is, the longer the convergence takes. Indeed, at each iteration, the robot must stop to measure the distance to the obstacle ahead. We chose a time of 1 second (which can be adjusted ; smaller values risk noisy measurements due to fewer samples, and larger values significantly increase total movement time) for data acquisition. The algorithm takes about 1.1 seconds per cycle (data acquisition plus distance measurement). Therefore, a very small t' leads to many repetitions of the last two steps, which is not practical. We selected $t' = 0.4\,s$. For the average robot speed used in the calculation of t, we overestimated it to $0.5\,m/s$.

With these parameters, we achieve a positioning precision of approximately 5 cm for the robot's forward movement.

Parameter	**Definition**	**Impact / Considerations**
d	Target distance to move forward	Distance to be reached
d'	Sufficient initial move distance	Must be less than or equal to d, depends on ϵ to avoid overshooting d on first move
t	Move time corresponding to d'	Calculated as $t = \frac{d}{v+\epsilon}$, ensures speed overestimation to avoid overshoot
t'	Small incremental move time for fine adjustment	Smaller t' increases precision but lengthens total duration due to multiple measurements
ϵ	Margin added to average speed v	Smaller $\epsilon \to 0$ means $d' \to d$ from below ; requires careful tuning to avoid overshoot
Measurement cycle time	Time to acquire and process lidar data	About 1.1 s ; limits how small t' can realistically be
Average speed v	Estimated robot speed for computing t	Overestimated here as $0.5\,m/s$ to ensure safety and precision
Final precision	Accuracy of the final traveled distance	Approximately 5 cm with chosen parameters ; strongly depends on t' and ϵ

Table 5.1 – Choice of parameters and their impact on robot forward movement

5.7.1.1 Distance measurement

To calculate the distance to the obstacle in front of the robot, the lidar mounted on the robot scans the environment and returns the distance corresponding to the angle closest to 0°, which represents the direction directly in front of the robot.

We attempted to significantly improve this algorithm (in order to reduce its duration and the number of iterations) based on feedback from various project contributors. We designed a new version that allows the robot to move in real-time while continuously measuring the distance to obstacles. This approach enables the robot to stop as soon as the specified distance has been exceeded.

However, this solution proved inconclusive due to our current Lidar implementation. The Lidar requires approximately 1 second to scan the environment and return the distance to the obstacle in front. During this

interval, the robot continues to move, and by the time the Lidar result is available, it is already too late to stop in time.

One idea to overcome this limitation would be to run the Lidar on a separate core of the Raspberry Pi. It would continuously scan the environment, and whenever the main algorithm needs the distance to the front obstacle, it would simply use the most recent data collected over the past 1 second. We estimate the execution time of this improved approach to be nearly 0.1s, as the Lidar data would already be available when needed which avoids the data acquisition.

5.7.2 Turn + angle

We were not able to get this algorithm to work properly, as we could not obtain an accurate measurement of the robot's rotation angle using the onboard lidar (see below).

Let's now consider the task of rotating the robot by an angle θ.

1. The lidar scans the environment to determine the robot's orientation relative to fixed obstacles.
2. If the current orientation is less than θ, the robot rotates for a sufficient duration t.
3. After this rotation, the new orientation is estimated.
4. If the angle is still insufficient, the robot rotates again for a shorter duration t'.
5. These last two steps are repeated until the robot has reached or slightly exceeded the target angle θ.

The goal is to approach the desired angle θ as closely as possible :

- During the first rotation, we use a sufficiently long duration t so that the robot comes close to θ without exceeding it. - Then, in the following iterations, we use a short duration t' so the robot rotates incrementally and ends up just exceeding θ by a minimal amount.

To estimate the initial duration t, we used :

$$t = \frac{\theta}{\omega}$$

where ω is the estimated angular speed of the robot. We major the angular speed of the robot by 180 $/s$, which gives :

$$t = \frac{\theta}{180}$$

In practice, we set $t' = 0.3\,s$, which provides a good compromise between speed and accuracy for the following iterations.

Parameter	**Definition**	**Impact / Considerations**
θ	Target angle to rotate	Angle to be reached
t	Initial rotation time	Approx. θ/ω ; assumes constant angular speed
t'	Small rotation time for fine adjustment	Improves final accuracy ; smaller t' increases precision but lengthens convergence time
ω	Estimated angular speed	Overestimated to avoid overshoot ; assumed to be $180°/s$
Lidar scan time	Time to acquire and process lidar data	4 s ; limits how fast feedback-based correction can be applied
Real-time feasibility	Whether measurements can be taken while rotating	Not feasible in current setup due to 4s scan delay ; could be solved with parallel processing
Final precision	Accuracy of final angle reached	Depends on t', ω, and scan delay

Table 5.2 – Choice of parameters and their impact on robot rotation

5.7.2.1 Measurement of the angle of rotation (First method)

The goal of the above algorithm is to accurately measure the robot's rotation angle. Our first approach was to compute the average position (centroid) of all lidar points in the environment before the rotation. Then, after the robot had rotated, we scanned the environment again and computed the new centroid. The estimated rotation angle was then calculated as the angular difference between the two centroid vectors.

This algorithm works well when the lidar is placed on the ground and the rotation is performed very carefully to avoid any translation of the lidar during the turn. However, in practice, the lidar is mounted on the robot.

We tried placing the lidar on the robot's rotation axis to minimize translational effects, but this axis is not fixed or perfectly stable. This leads to completely unreliable rotation angle measurements due to two main factors :

— The appearance of new points in the lidar scan.

— Changes in the distance to existing points, as the robot moves closer to some and farther from others between rotations, because the rotation axis of the robot is not constant.

A simple and exaggerated example helps illustrate the first point : Imagine that at $t = 0$ s, the robot is facing a box, directly in front of face A. The lidar calculates the centroid based solely on points from face A. Then, the robot translates slightly to the left or right, revealing a new part of the box : face B or face D. The lidar now sees both face A and face D. As a result, new points from face B or face D appear, which significantly distort the centroid calculation and thus corrupt the angle measurement.

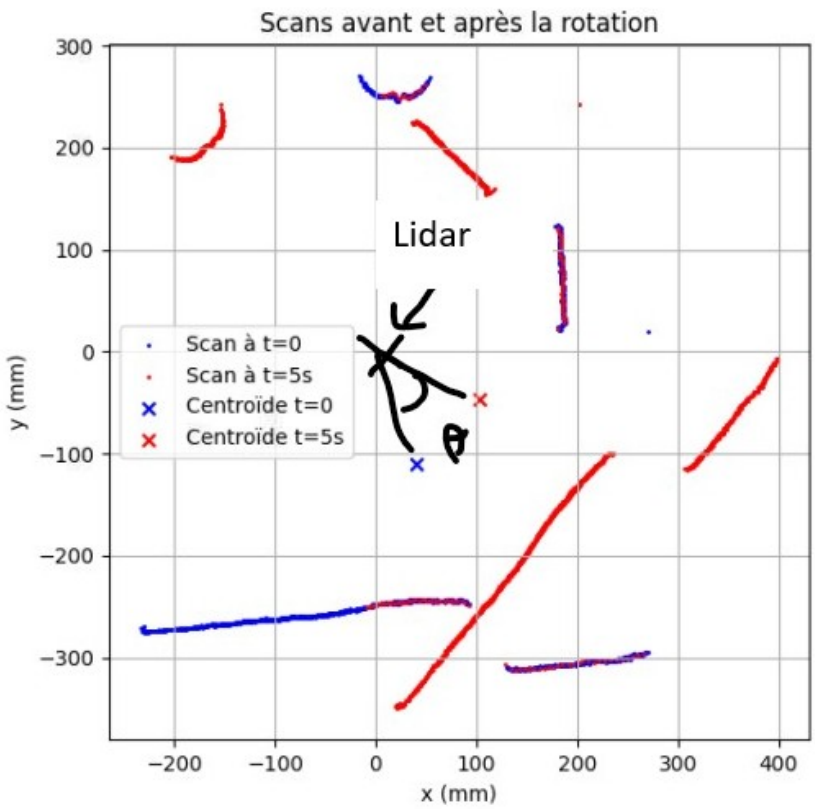


FIGURE 5.12 – 1st method for the measurement of the angle of rotation associated to the 5.11

We obtained the figure by scanning this environment at $t = 0$, then rotating the LiDAR, and finally performing a new scan at $t = 5$ s. We obtained an estimated rotation angle of $\theta = 45.56°$.

The algorithm with the lidar placed on the ground in such a way that there is no translation during rotation achieves a maximum precision of about 1°. We measured this accuracy using a protractor during a 90° rotation and averaged several measurements to obtain this result. The achieved precision depends on several factors, such as the scanning distance and the duration of the scan. We chose a scanning distance of 2 meters and a duration of 4 seconds. This allows for a sufficient number of points to be collected for each centroid calculation.

5.7.2.2 Measurement of the angle of rotation (Second method)

The idea now is to accurately map the environment with obstacles, compute the centroids of these obstacles, and use them to estimate both the rotation and small translations after an operation involving the lidar. Indeed, the lidar becomes much less sensitive to minor translations since the centroids of the obstacles remain almost unchanged. However,

we have not yet had time to perform actual measurements to evaluate the precision of this algorithm and to quantify the differences compared to the previous algorithm.

We started from an initial algorithm for obstacle detection based on the construction of convex hulls. The principle is as follows : each point in the LiDAR point cloud is associated with a ball of radius ε centered at that point. If two such balls intersect, their corresponding points are considered connected. This allows us to identify connected groups of points, for which we can then build the convex hull — the smallest convex polygon enclosing all points in the group.

Originally, this algorithm aimed to prevent the robot from attempting to pass between two pillars when the gap between them was smaller than the robot's diameter d. To achieve this, we set $2\varepsilon = d$, so that if two pillars were too close, their LiDAR points would become connected and form a single convex hull interpreted by the robot as an impenetrable wall.

This principle then inspired us to use convex hulls for detecting the robot's movements within its environment. The idea is to construct convex hulls from connected point clusters whose size (cardinality) lies between $n_{\min}$ and $n_{\max}$, with $n_{\max} > n_{\min}$. For each hull, we compute its barycenter, which serves as a representative landmark point of the environment.

The parameters $n_{\min}$ and $n_{\max}$ are crucial : $n_{\min}$ must be large enough to ensure that barycenters are meaningful (not resulting from small noisy artifacts), while $n_{\max}$ must remain small enough to generate a reasonable number of barycenters, thus providing rich coverage of the scene.

We then fix a time interval $\Delta t > 0$. A set of barycenters is generated at time t_1, and another set at time $t_2 = t_1 + \Delta t$.

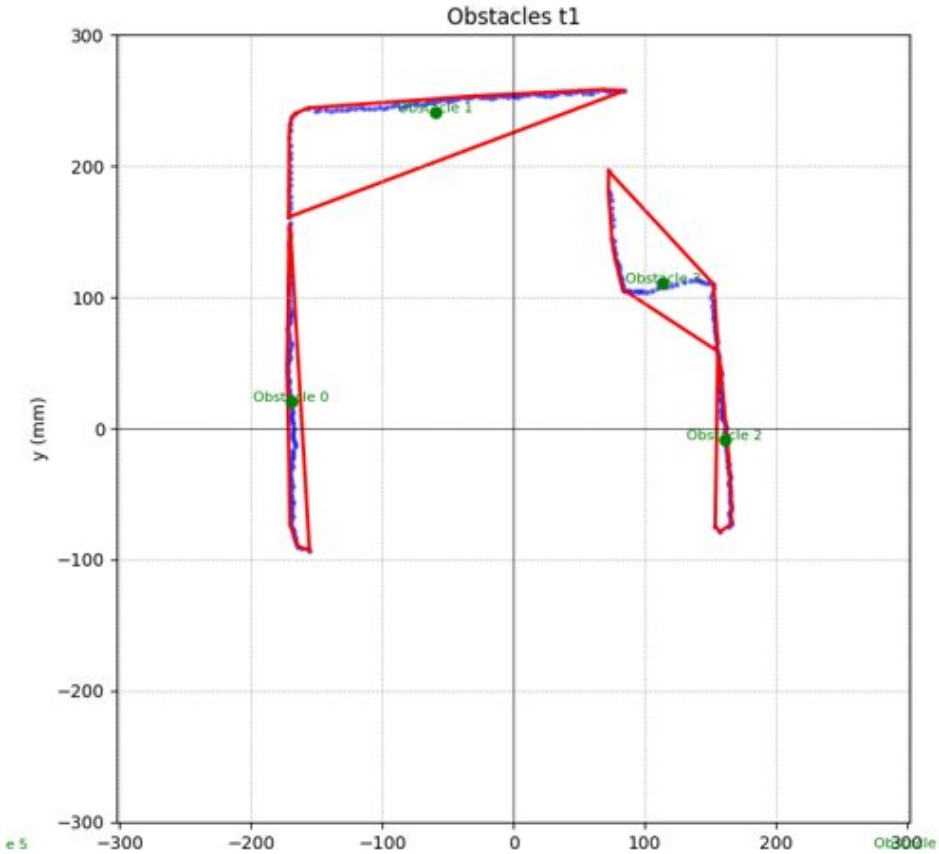


FIGURE 5.13 – Obstacle detection at time t_1

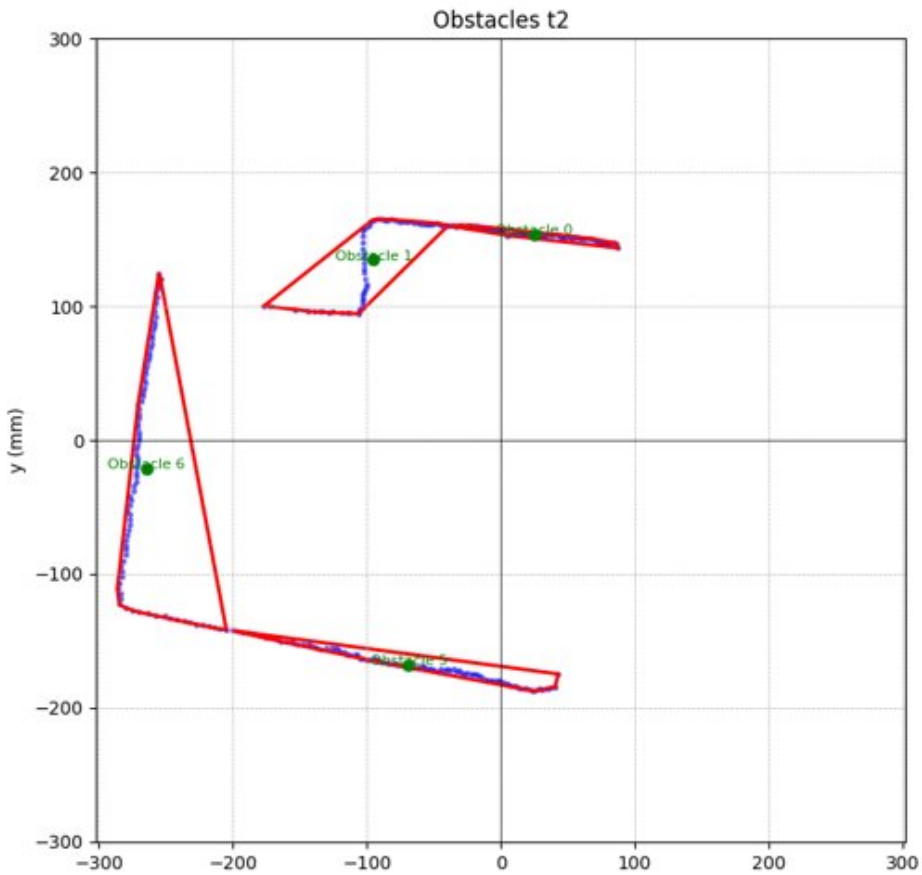


FIGURE 5.14 – Obstacle detection at time t_2

The robot is initially located at the point $(0, 0)$ and is facing the axis $x = 0$ (the y-axis). At time t_2, the robot has performed a rotation. On the plot at t_2, the robot is still located at the point $(0, 0)$ and continues to face the axis $x = 0$ (the y-axis).

The transformation undergone by the barycenter coordinates can be modeled by the equation :

$$x \leftarrow Rx + t$$

where R is a planar rotation matrix and t is a translation vector.

The main challenge is to correctly associate each point detected at time t_2 with its counterpart at time t_1 in order to solve the transformation system. Note that the LiDAR may detect additional points at t_2, potentially resulting in a strictly larger number of barycenters.

However, as $\Delta t \to 0$, the sets of points detected at t_1 and t_2 tend to coincide. Small variations in detected points have limited impact : for example, newly detected points in a region may simply be absorbed into an existing convex hull without significantly shifting its barycenter due to their low relative weight.

The matching method used relies on nearest-neighbor search via a KD-tree to associate points detected at two different time steps.

We tested this algorithm on the LiDAR without the robot, and it works correctly for translations up to 1 cm. We have not yet had time to precisely evaluate the accuracy of the rotation. However, a translation on the order of 1 cm corresponds exactly to the range of motion of the robot's rotation axis. These results are therefore very promising for achieving precise angle rotations with the LiDAR mounted on the robot.

5.7.3 Translation to the right or left by a given distance

Furthermore, this rotation estimation algorithm was developed towards the end of the project, and as such, we did not have enough time to test it on the actual robot. This means that although we have a theoretical algorithm to rotate by a precise angle, it is not yet operational, since the first version of the angle estimation process is only implemented using the LiDAR mounted on the robot. As a result, we currently cannot perform actual rotations.

However, our objective remains to move the robot from a point A to a point B while avoiding obstacles. Realizing that precise rotations were not immediately feasible, we reconsidered the role of rotation in navigation. In our scenario, rotations are mainly useful to enable lateral translations — for instance, turning 90° to the right, moving forward, and turning 90° back to the left.

To replace this rotation-based approach, we leveraged the ability to control the robot's wheels independently. This capability, developed by previous teams, allows the robot to translate directly to the right or left without changing its orientation.

Building on the same principle we used for forward motion — i.e., moving a controlled distance using time and velocity — we implemented lateral translation (right or left) by small increments. For example, to translate to the right by a certain distance, the robot first moves slightly right, then uses the LiDAR to measure the distance to the obstacle on that side. If the obstacle is still within range, it continues to shift incrementally, re-measuring after each short movement, until the desired clearance is achieved.

This iterative approach effectively simulates lateral displacement without requiring rotational motion. It has proven to be a practical and robust workaround, allowing the robot to bypass obstacles and continue toward its destination even without rotation.

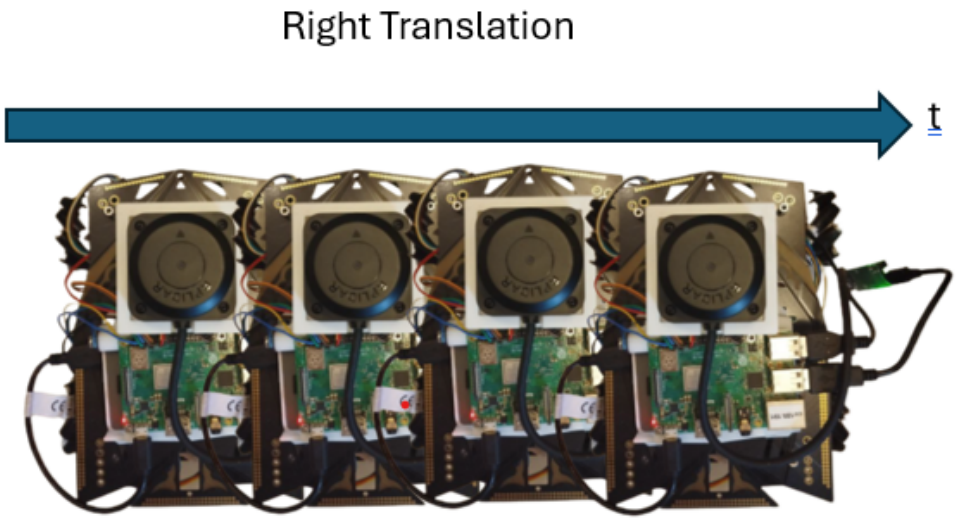


FIGURE 5.15 – Right translation of the robot

5.8 Obstacle avoidance with lidar and the first steps of an Autonomous Navigation

When the robot detects an obstacle in front of it that prevents it from moving forward by a distance d, it measures the width of the obstacle by calculating the angle between its two edges and using the known distance between itself and the obstacle. It then shifts laterally by a distance d' that is strictly greater than the sum of the obstacle's width and its own diameter. After that, it moves forward by the initial distance d, maintaining the same direction, while keeping in memory that once this movement is completed, it must return to its original trajectory by shifting back by the same distance d'. The same algorithm is then repeated.

In the following figure, d = 1.54 meters and d' = 0.5 meter.

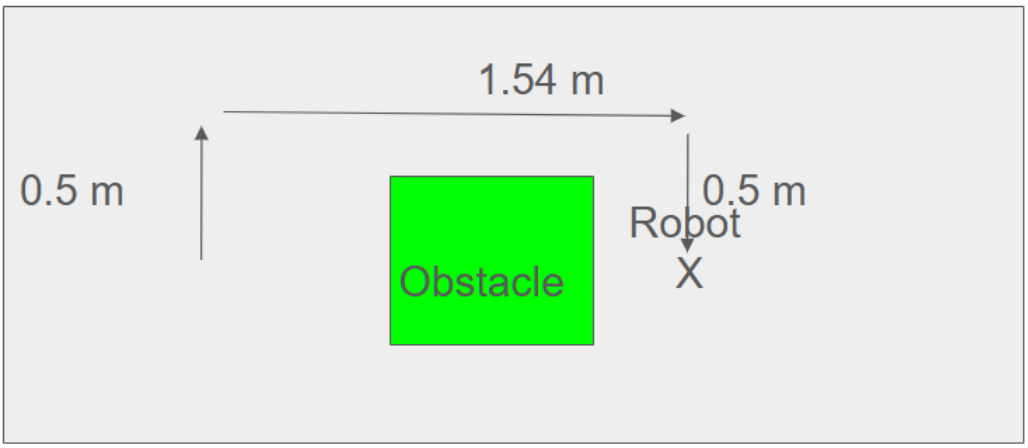


FIGURE 5.16 – Obstacle avoidance (the robot is always in the axis of the obstacle)

5.9 Conclusion of this part : Towards a Robust LiDAR-Based SLAM for Autonomous Navigation

SLAM, or *Simultaneous Localization and Mapping*, is a fundamental method in robotics that enables a robot to localize itself in an unknown environment while simultaneously building a map of that environment in real time.

The main challenge we face is related to the robot's translations : with each movement, new obstacle points appear in the LiDAR data, modifying the configuration of detected clusters. This variation makes it difficult to reliably match landmarks detected at different moments, thus complicating the accurate estimation of the robot's pose (position and orientation). This issue currently hinders the implementation of a stable and robust SLAM algorithm.

Despite this challenge, we have developed several fundamental building blocks :

- A method that allows the robot to move forward by a precise distance, based on repeated LiDAR measurements combined with time/speed control, achieving an accuracy of approximately 5 cm.
- A rotation algorithm that enables the robot to turn by a given angle, using an initial estimation of rotation duration followed by iterative adjustments to reach the target angle with good accuracy.
- An obstacle detection approach that groups LiDAR points into clusters, then constructs convex hulls around these clusters, providing a clear and structured representation of obstacles as convex polygons. This allows us to extract barycenters that serve as reference points.
- A model for the transformation experienced by the environment (a combination of rotation and translation) based on these barycenters, allowing us to accurately estimate the robot's movement. Currently, this model works for translations of up to 1 cm.

We tested this transformation model using only the LiDAR (without the robot), and it performs well for maximum translations of 1 cm. We have not yet had time to precisely evaluate the accuracy of the rotation. However, a 1 cm translation corresponds exactly to the amplitude of the rotation axis movement of the robot. These results are therefore very promising for accurately turning by a given angle with the LiDAR mounted on the robot.

Thanks to these developments, our robot can now avoid obstacles and autonomously move from one point to another (from A to B). This current version already works but still requires improvements. So far, we have used only lateral translations to reorient the robot, since the rotation algorithm was not yet fully implemented. Now, with this new method of estimating rotation using LiDAR data, we can integrate actual rotations into the robot's motion.

The next step is to test this rotation method with the LiDAR mounted on the robot, validate the rotation by a given angle, and finalize a complete navigation algorithm capable of using both translations and rotations while avoiding obstacles.

Ultimately, the integration of these components paves the way for developing our own SLAM algorithm. By accurately determining the transformations between each LiDAR scan, the robot will be able to reconstruct a coherent map of its environment while dynamically localizing itself within it.

However, the robot only works with obstacles that are higher than the height of the lidar, i.e. the height of the robot, i.e. 20 cm. Furthermore, the obstacles must be fixed, otherwise the measurements will be distorted.

This work brings us significantly closer to our objective : enabling the robot to move autonomously and intelligently in an unknown environment, thus fully addressing the initial challenge.

Chapitre 6

Conclusion

6.1 Conclusion of the report

Throughout this project, we significantly enhanced the robot's mobility, sensing, and autonomous navigation capabilities. We refined motor control using nonlinear feedback strategies, addressed sensor latency issues, and developed a current probe system to better estimate torque and friction effects. The integration of a LiDAR sensor allowed for accurate obstacle detection and the first steps toward real-time environmental mapping. Our work on landmark detection—via both RANSAC and convex hulls—laid the foundation for precise localization. Additionally, we implemented basic yet robust movement primitives and an obstacle avoidance strategy that simulates lateral motion without requiring exact rotation. These developments collectively move the platform closer to achieving LiDAR-based SLAM, with further work needed on reliable landmark association under translation. Overall, our contributions reinforce the platform's potential for autonomous navigation in real environments.

6.2 Remaining challenges

OBJECTIVES	**Problems Encountered**	**CONCLUSION**	**Perspectives**
Improve the speed control system	- Nonlinear behavior makes control difficult - Trajectory deviations due to surface characteristics of the ground	- Sensors improved and verified - Deviation remains significant during movement	- Finalize feedback control using other strategies
Develop wheel position control	- Strong nonlinear behavior of motors	- Electrical and mechanical modeling of the motors	- Finalize feedback control using other strategies
Integrate a LiDAR into the robot to improve obstacle detection and trajectory planning	- Time and usage constraints of the robot - Code integration in Python and C++ - Localize the robot in space - Robot's rotation axis not constant (new points appear for angle calculation)	- Successful LiDAR software integration on Raspberry Pi with clear readme for the following groups - Successful physical integration of LiDAR - Progress in developing a robot localization strategy - Code to move with given distances while avoiding obstacles	- Speed up the algorithm to allow faster translation - Refine SLAM method development from previous work - Use a compass sensor to measure rotation angle

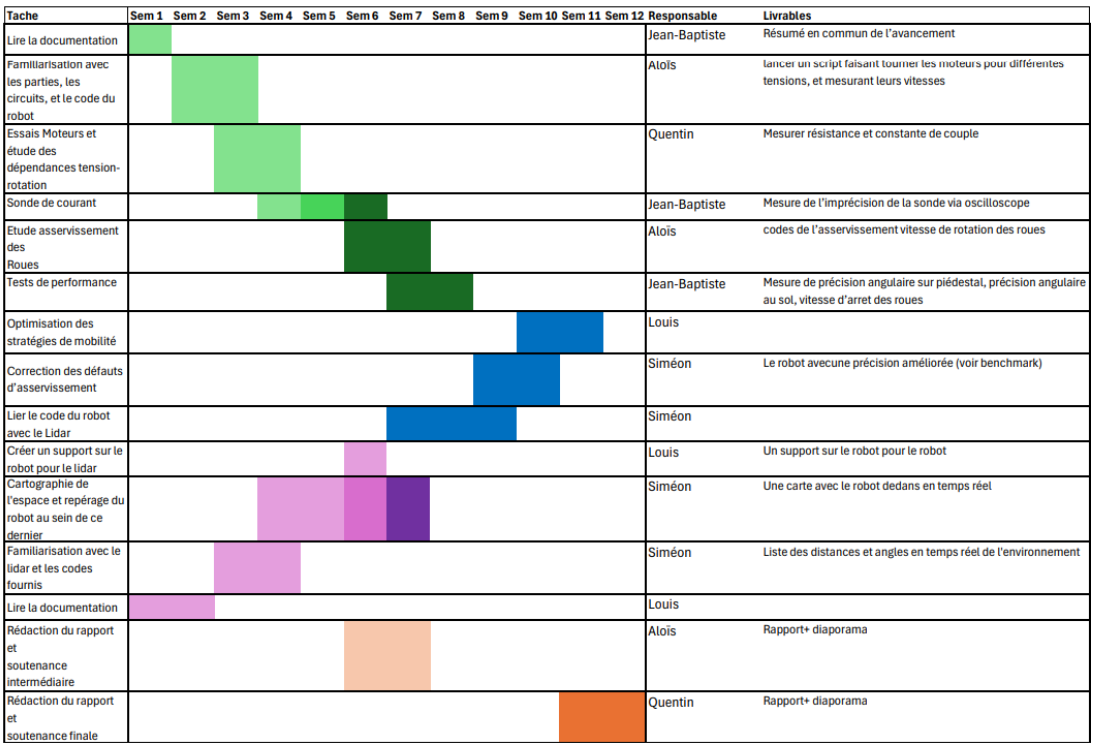


FIGURE 6.1 – Project planning and current progress.

As shown in Figure 6.1, the project planning and its progress are illustrated.

Bibliographie

[1] Louis Aubrun, Alex García Borrut, Luis-López Pardo Ferrer, Alisson Nunes Bonatto, Romero Perardt, Magalhães Brito, and Eduardo Rosa de Lima. Pôle projet : Mobile, 2024.